ALARM CLOCK

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

MONISHA RAGHINI CHELLE	2520030537
SHAIK SABIYA TABASSUM	2520030502
K. SIRI CHANDANA	2520030453

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"ALARM CLOCK"** is a record of Bonafide work of **MONISHA RAGHINI CHELLE - 2520030537, SHAIK SABIYA TABASSUM-2520030502, K. SIRI CHANDANA - 2520030453** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

MONISHA RAGHINI CHELLE 2520030537
SHAIK SABIYA TABASSUM 2520030502
K. SIRI CHANDANA 2520030453

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project-based report entitled **"ALARM CLOCK"** is a bonafide work done and submitted by **MONISHA RAGHINI CHELLE - 2520030537, SHAIK SABIYA TABASSUM- 2520030502, K. SIRI CHANDANA - 2520030453**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. K. Anusha, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

MONISHA RAGHINI CHELLE 2520030537
SHAIK SABIYA TABASSUM 2520030502
K. SIRI CHANDANA 2520030453

Abstract

Alarm Clock is a Linux-based terminal application developed in the C programming language. The application provides a simple menu-driven interface through which the user can set an alarm, view the configured alarm, cancel it, or exit the program.

The project is designed to demonstrate operating-system and systems-programming concepts in one integrated application. CO1 reports process and operating-system information using getpid(), getppid(), uname(), and write(). CO2 demonstrates process creation and control using fork(), kill(), and waitpid(). CO3 implements alarm scheduling and asynchronous notification using alarm(), SIGALRM, sigaction(), and sigwait().

Additional modules are integrated through alarm.c. CO4 manages the alarm structure using dynamic memory allocation and release. CO5 stores, loads and removes alarm information using Linux file I/O. CO6 provides a POSIX monitor thread using mutex and condition-variable synchronization. The project is compiled with GCC in Ubuntu/WSL and executed as a single terminal application

Table Of Contents

ACKNOWLEDGEMENT 4
Abstract 5
CHAPTER - 1 INTRODUCTION 11
1.1 Background of the Project 11
1.2Problem Statement 11
1.3 Scope of the Project 12
1.4 Objectives of the Project 13
1.5 Importance of the Application 13
1.6 Target Users / Audience 14
1.7 Motivation of the Project 14
1.8 Need for the Project 14
1.9 Proposed Solution 15
1.10 Project Workflow 15
1.11 Organization of the Project 17
1.12 Expected Outcome 17
CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY 18
2.1 Operating System Concepts Related to the Project 18
2.1.1 Operating System Services 18
2.1.2 Process Management 18
2.1.3 Signals and Asynchronous Notification 19
2.1.4 Dynamic Memory Management 19
2.1.5 File Input and Output 19
2.1.6 Threads and Synchronization 19
2.1.7 Integration of Operating System Concepts 20
2.2 Existing Methods / Algorithms 20
2.2.1 Comparison of Polling and Timer-Based Operation 21
2.3 Proposed Approach 21
2.3.1 Alarm Input and Validation 23
2.3.2 Alarm Delay Calculation 23
2.3.3 Signal Notification 23
2.3.4 Alarm Trigger Processing 23
2.3.5 Resource Cleanup 24

2.4 Advantages of the Proposed Approach 24
2.4.1 Uses Linux Timing Facilities 24
2.4.2 Demonstrates Signal Handling 24
2.4.3 Integrates CO1 to CO6 24
2.4.4 Simple Terminal-Based Interface 24
2.4.5 Separation of Responsibilities 25
2.4.6 Demonstrates Resource Management.....25
2.4.7 Demonstrates Multiple Execution Mechanisms 25
2.4.8 Suitable for Linux-Based Demonstration 25
2.5 Background Study of Linux and POSIX Environment 25
2.6 Relationship Between the Concepts 26
2.7 Summary of the Background Study.....27
CHAPTER - 3 SYSTEM REQUIREMENTS 28
3.1 Hardware Requirements 28
3.2 Software Requirements.....28
3.2.1 Windows 28
3.2.2 Ubuntu through WSL 29
3.2.3 Visual Studio Code 29
3.2.4 GCC 29
3.2.5 Git 29
3.3 Operating System Environment – Ubuntu / WSL 29
3.3.1 Role of WSL in the Project.....31
3.3.2 Terminal-Based Execution 31
3.3.3 Compilation and Execution Environment 31
3.4 Development Tools and Compilers 32
3.4.1 Source-Code Editor 32
3.4.2 Compiler 32
3.4.3 Project Compilation Structure 32
3.4.4 Development Workflow 34
CHAPTER - 4 TECHNOLOGY AND TOOLS USED 35
4.1 C Programming.....35
4.2 GCC Compiler.....36
4.3 Linux System Calls and APIs 37
4.3.1 Process Identification APIs.....37
4.3.2 Operating-System Information 37
4.3.3 Process Creation and Control 37
4.3.4 Alarm and Signal APIs 38

4.3.5 File APIs 39
4.3.6 Thread APIs 39
4.4 POSIX Threads / Synchronization 39
4.4.1 Monitor Thread 40
4.4.2 Mutex 40
4.4.4 Thread Completion 41
4.5 Linux Commands and Utilities 41
4.5.1 wsl 41
4.5.2 cd 41
4.5.3 ls 42
4.5.4 gcc 42
4.5.5 ./alarm_clock 42
4.5.6 Relationship Between Commands 42
4.6 Other Tools / Libraries Used 43
4.6.1 Overall Technology Stack 43
4.6.2 Summary 44
CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN 46
5.1 Overall System Architecture 46
5.2 System Workflow 47
5.3 Flowchart / Data Flow 49
5.4 Module Description 51
5.5 Input and Output Design 51
CHAPTER - 6 IMPLEMENTATION 53
6.1 Module-wise Implementation 53
6.1.1 main.c 53
6.1.2 alarm.c 53
6.1.3 CO1 – co1.c 54
6.1.4 CO2 – co2.c 55
6.1.5 CO3 – co3.c 55
6.1.6 CO4 – co4.c 56
6.1.7 CO5 – co5.c 57
6.1.8 CO6 – co6.c 57
6.2 Algorithm / Pseudocode 58
6.3 Core C Implementation 59
6.4 Linux System Calls / APIs Used 59
6.5 Process / Thread Implementation 60
6.6 IPC / Synchronization Implementation 61

6.7 Memory Management Implementation 62
CHAPTER - 7 EXECUTION AND DEMONSTRATION 63
7.1 Compilation in Ubuntu 63
7.2 Execution Commands 63
7.3 Sample Inputs and Outputs 63
7.4 Screenshots of Execution 64
7.5 Demonstration of OS Concepts 67
7.6 Known Implementation Limitations 67
CHAPTER - 8 CONCLUSION 68
8.1 Summary of the Project 68
8.2 Objectives Achieved 69
8.2.1 Terminal-Based Alarm Interface 69
8.2.2 Alarm Scheduling 69
8.2.3 CO1–CO6 Integration 69
8.2.4 Process Management 69
8.2.5 Dynamic Memory Management 70
8.2.6 File Storage 70
8.2.7 Thread Synchronization 70
8.3 Results Achieved 70
8.4 Skills and Learning Outcomes 71
8.4.2 C Modular Programming 72
8.4.3 System Calls and APIs 72
8.4.4 Signals and Timers 72
8.4.5 Process Management 72
8.4.6 Memory and File Management 73
8.4.7 Thread Synchronization 73
8.5 Overall Conclusion 73
CHAPTER - 9 REFERENCES 74
9.1 Books 74
9.2 Research Papers 74
9.3 Linux / GNU Documentation 74
9.3.1 GCC Documentation 75
9.3.2 Linux Process Documentation 75
9.3.3 Linux Signal and Alarm Documentation 75
9.3.4 Linux File I/O Documentation 76
9.3.5 POSIX Thread Documentation 76
9.4.1 Project Source Repository 76

9.4.2 Project README and Source Files 76
9.4.3 Project Execution Screenshots.....77
9.6 Reference Usage in the Project.....78
9.7 Summary of References.....79

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Alarm scheduling is a simple but useful example for demonstrating operating-system services. A user gives a target time, the application calculates the waiting duration, and the operating system provides a notification mechanism when the timer expires. This project implements that idea as a terminal application in C for a Linux environment.

An alarm clock is a time-based application in which the system must continuously maintain awareness of the current time and respond when a user-defined target time is reached. In a conventional application, the program could repeatedly check the current time. However, such an approach would require continuous polling and would not clearly demonstrate the operating-system mechanisms used for event notification. The proposed project instead uses Linux timer and signal facilities to demonstrate how an application can wait for an operating-system event.

The Alarm Clock project is designed as a menu-driven terminal application. The user can select an operation from the displayed menu and provide the required alarm information. The application accepts an hour, minute, second and label, validates the entered values, calculates the required waiting duration and schedules the alarm.

The project is implemented in the C programming language because C provides direct access to Linux and POSIX APIs. Functions such as getpid(), getppid(), uname(), fork(), kill(), waitpid(), alarm(), sigaction(), open(), read(), write(), close(), unlink() and POSIX thread functions can be used to demonstrate operating-system concepts in a practical manner.

The project is divided into multiple source files. The main.c file provides the application entry point and menu flow, while alarm.c acts as the central coordination module. The CO1 to CO6 modules implement different operating-system concepts and are integrated into the same Alarm Clock application.

The project therefore serves two purposes. First, it provides the basic functionality of a terminal alarm clock. Second, and more importantly for the Operating Systems and Systems Programming course, it provides a practical demonstration of how a user-space C application interacts with Linux services.

1.2Problem Statement

The project requires a small terminal application that can accept an alarm time, maintain the configured alarm, notify the user when the time is reached, and allow the alarm to be viewed or cancelled. At the same time, the application should demonstrate course concepts such as system calls, signals, process management, memory management, file operations and POSIX synchronization.

A simple alarm application may appear to be a basic time-management program, but implementing it at the operating-system level introduces several important problems. The application must determine the current system time, accept and validate user input, calculate the difference between the current time and the requested alarm time, and wait for the required duration.

The program must also respond correctly when the alarm time is reached. Instead of continuously checking the time, the project uses Linux's alarm and signal mechanism. The alarm() function schedules a timer and the operating system generates SIGALRM when the timer expires. The project then handles this event and performs the required alarm operations. Another requirement is to demonstrate process management. The project creates a child process when the alarm is triggered and demonstrates process termination and waiting using fork(), kill() and waitpid().

The application also has to manage resources correctly. Dynamically allocated alarm information must eventually be released. Alarm information is stored in a file using Linux file operations, and the monitor thread must be properly synchronized and joined before resources are released. Therefore, the main problem addressed by this project is the design and implementation of a functional terminal alarm application while integrating multiple Linux operating-system concepts into one coordinated program.

1.3 Scope of the Project

The scope is a single Linux terminal application. The user interacts with the application through a text menu. The application supports setting an hour, minute, second and label; viewing the active alarm; cancelling the alarm; and exiting while releasing allocated resources.

The project focuses on user-space Linux programming and does not require a graphical user interface. All interaction takes place through the terminal. This makes the operating-system concepts visible through terminal output and allows the execution of system calls, signals, processes and threads to be observed.

The major functional scope includes:

- Setting an alarm using hour, minute and second.
- Assigning a label to the alarm.
- Validating the entered time.
- Calculating the waiting duration.
- Scheduling the Linux alarm timer.
- Handling the SIGALRM notification.
- Viewing the currently configured alarm.
- Cancelling an active alarm.
- Storing alarm information in data/alarms.txt.
- Creating and controlling a child process.
- Managing dynamically allocated alarm memory.
- Using a POSIX monitor thread for alarm-state monitoring.
- Releasing resources when the alarm is completed, cancelled or the application exits.

The project is primarily intended for academic demonstration. It does not attempt to replace a full desktop or mobile alarm application.

The current project focuses on a single alarm at a time. Features such as multiple simultaneous alarms, graphical notifications, mobile notifications and cloud-based synchronization are outside the current scope.

1.4 Objectives of the Project

The main objectives of the Alarm Clock project are:

- Provide a simple terminal interface for alarm management.
- Calculate the time difference between the current time and the requested alarm time.
- Schedule an operating-system timer using alarm().
- Handle SIGALRM and notify the application when the timer expires.
- Demonstrate Linux process creation and control.
- Demonstrate dynamic memory, file I/O and POSIX thread synchronization.
- Release alarm, thread, timer and file resources during cancellation and shutdown.
-

In addition to these functional objectives, the project aims to provide practical understanding of Linux systems programming. Each course outcome is connected to a meaningful part of the application rather than being implemented as an unrelated standalone program.

CO1 demonstrates Linux operating-system services through process identification, operating-system information and direct system-call output.

CO2 demonstrates process management by creating and controlling a child process during the alarm event.

CO3 demonstrates signal-based alarm scheduling using the Linux timer and SIGALRM.

CO4 demonstrates dynamic memory management by allocating and releasing the alarm structure.

CO5 demonstrates Linux file I/O by storing and removing alarm information from the project data file.

CO6 demonstrates POSIX threads and synchronization through a monitor thread, mutex and condition variable.

Thus, the overall objective is to combine these concepts into one working application and provide a practical understanding of how different operating-system facilities can cooperate within a C program.

1.5 Importance of the Application

The project connects multiple operating-system concepts to one practical program. Instead of demonstrating each concept as an isolated program, the CO modules are integrated through alarm.c and are activated at different points of the alarm workflow.

The project is important from a systems-programming perspective because it demonstrates the interaction between an application and the Linux operating system. The application does not simply calculate and display a time value. It uses operating-system services for process information, timers, signals, processes, file operations and thread synchronization.

The alarm timer demonstrates how an application can request a service from the operating system and later receive a notification. The process-control component demonstrates the relationship between a parent process and a child process. The file-management component demonstrates how application data can be stored outside the program's memory. The memory-management component demonstrates controlled allocation and release of resources.

The project also demonstrates the importance of synchronization when multiple execution contexts access shared information. The monitor thread uses synchronization mechanisms to safely observe the alarm state and respond to the alarm event.

From an educational perspective, the application provides a common scenario through which several different OS topics can be understood together. Students can observe how processes, signals, memory, files and threads are not independent concepts but can work together within a single application.

1.6 Target Users / Audience

The application is primarily intended for academic demonstration and learning of Linux systems programming. It can be used by students to understand how a user-space C program communicates with Linux through system calls, signals, processes, files and threads.

The main target audience includes students studying Operating Systems, Systems Programming, Linux programming and C programming. The project can also be used by instructors as a simple demonstration of operating-system concepts.

The terminal interface makes the application easy to execute and observe in a Linux environment. The output provides visible information about the process ID, operating system, alarm scheduling, signal reception, process control and resource cleanup.

1.7 Motivation of the Project

The main motivation for developing the Alarm Clock project is to connect theoretical operating-system concepts with practical implementation. Operating-system topics such as processes, signals and threads can be difficult to understand when studied only through definitions. A working application provides a direct way to observe these concepts during execution.

An alarm clock provides a suitable example because its basic operation naturally involves time measurement, waiting, event notification and user interaction. These requirements can be mapped to Linux operating-system facilities.

The project was therefore selected to provide a simple application with enough operating-system interaction to demonstrate the course outcomes in a meaningful way.

1.8 Need for the Project

A practical systems-programming project is useful for understanding how software communicates with the operating system. The Alarm Clock project provides such an environment by combining several OS mechanisms.

There is a need for the application to determine the current system time accurately, calculate a future target time, wait without unnecessary continuous processing, receive an operating-system notification, and perform the required action when the alarm occurs.

There is also a need to demonstrate proper resource handling. Processes, threads, dynamically allocated memory, timers and file descriptors should not be left active unnecessarily. Therefore, the project includes cancellation and shutdown operations that release resources.

1.9 Proposed Solution

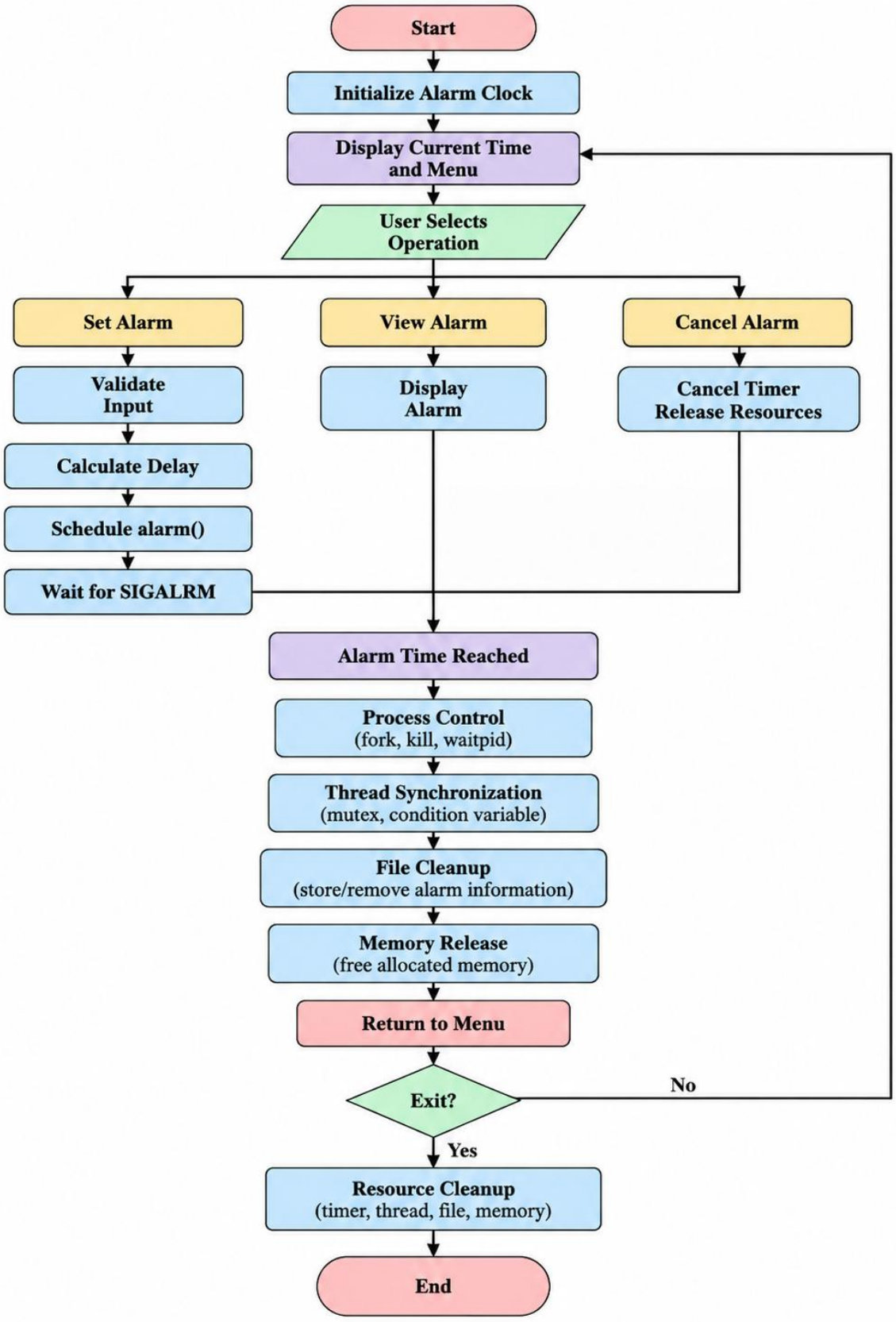
The proposed solution is a menu-driven C application executed in a Linux environment. When the application starts, it initializes the alarm system and displays the current time and menu. When the user selects **Set Alarm**, the application accepts the requested time and label. The entered values are validated before the alarm is created. The program calculates the difference between the current time and target time and schedules the timer.

The alarm mechanism uses Linux SIGALRM notification. A monitor mechanism is also used to coordinate the alarm state. When the alarm event occurs, the application displays the notification and performs the required process-control and cleanup operations.

The user can also select **View Alarm** to display the currently configured alarm or **Cancel Alarm** to stop an active alarm. When the user selects **Exit**, the application performs resource cleanup before terminating.

1.10 Project Workflow

The overall workflow of the Alarm Clock application can be summarized as follows:



This workflow demonstrates how the individual modules participate in the same application.

1.11 Organization of the Project

The project is organized into multiple C source and header files to separate different responsibilities.

main.c provides the entry point and menu-driven user interaction. alarm.c provides the central coordination of alarm operations. alarm.h contains the alarm data structure and alarm-related declarations. os_project.h contains the declarations required to integrate the CO modules.

The CO files implement individual operating-system concepts. co1.c handles OS services, co2.c handles process control, co3.c handles alarm and signal functionality, co4.c handles dynamic memory, co5.c handles file operations, and co6.c handles POSIX threads and synchronization. The separation into modules makes the project easier to understand, test and maintain while still producing a single executable Alarm Clock application.

1.12 Expected Outcome

The expected outcome of the project is a working Linux terminal application that allows the user to manage an alarm while demonstrating the required operating-system concepts.

After successful execution, the user should be able to set an alarm, view its details, cancel it when required and exit the application cleanly. During these operations, the application should demonstrate the corresponding CO modules and display useful execution information in the terminal.

The project is also expected to provide practical experience with Linux development using Ubuntu/WSL, GCC compilation, C programming, system calls, signals, processes, dynamic memory, file operations and POSIX threads.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The Alarm Clock project demonstrates several Linux and POSIX operating-system concepts through a single terminal-based C application. The main purpose of studying these concepts is to understand how an application interacts with operating-system services for timing, process management, signals, memory management, file handling, and thread synchronization.

The project is divided into CO1 to CO6 components. Each component demonstrates a particular operating-system concept, while the complete application integrates these concepts into the alarm scheduling workflow.

2.1.1 Operating System Services

CO1 demonstrates basic operating-system services used by the Alarm Clock application. The program obtains information such as the current process ID, parent process ID, operating-system information, and kernel release information.

The project uses system-level operations such as getpid(), getppid(), and uname() to obtain information about the running environment. The write() system call is also demonstrated as part of the OS service layer.

These operations show that a user-level C application can request services from the Linux operating system rather than performing every operation independently.

2.1.2 Process Management

CO2 demonstrates process control using Linux process-management facilities. A process represents a running instance of the Alarm Clock program or one of its related execution activities.

The project demonstrates the creation and management of a child process using fork(). The parent and child processes can be controlled using process-related operations. The kill() function is used for sending a signal to a process, while waitpid() is used for process synchronization and termination handling.

The process-control component becomes relevant when the alarm event is triggered. The application creates an alarm-related child process and demonstrates controlled termination after the alarm operation is completed.

2.1.3 Signals and Asynchronous Notification

CO3 demonstrates signals, particularly SIGALRM. Signals provide a mechanism through which the operating system can notify a process when a particular event occurs.

The application calculates the required waiting duration and uses alarm() to schedule the alarm. After the scheduled duration, the operating system generates SIGALRM.

The project uses sigaction() for signal handling. A background alarm-waiting mechanism waits for the alarm event and invokes the appropriate alarm-trigger processing when SIGALRM occurs.

This concept is important because the application does not need to continuously check the time in a busy loop. Instead, the operating system provides notification when the scheduled alarm event occurs.

2.1.4 Dynamic Memory Management

CO4 demonstrates dynamic memory management. The alarm information is managed using dynamically allocated memory during the operation of the application.

Functions such as malloc() and calloc() are used for dynamic memory allocation where required, and free() is used to release allocated memory.

Dynamic memory management demonstrates the responsibility of a C program to release resources after they are no longer required. In the Alarm Clock project, memory cleanup is performed as part of cancellation, alarm completion, and shutdown operations.

2.1.5 File Input and Output

CO5 demonstrates Linux file operations using the alarm data file data/alarms.txt.

The project uses file-related operations such as open(), read(), write(), close(), and unlink().

Alarm information can be stored in the file and later accessed by the application.

File handling provides persistent storage for alarm information during the execution workflow.

The project also demonstrates removal of the alarm file when the alarm-related resources are no longer required.

2.1.6 Threads and Synchronization

CO6 demonstrates POSIX threads and synchronization mechanisms. A monitor thread is created to coordinate alarm-related activity.

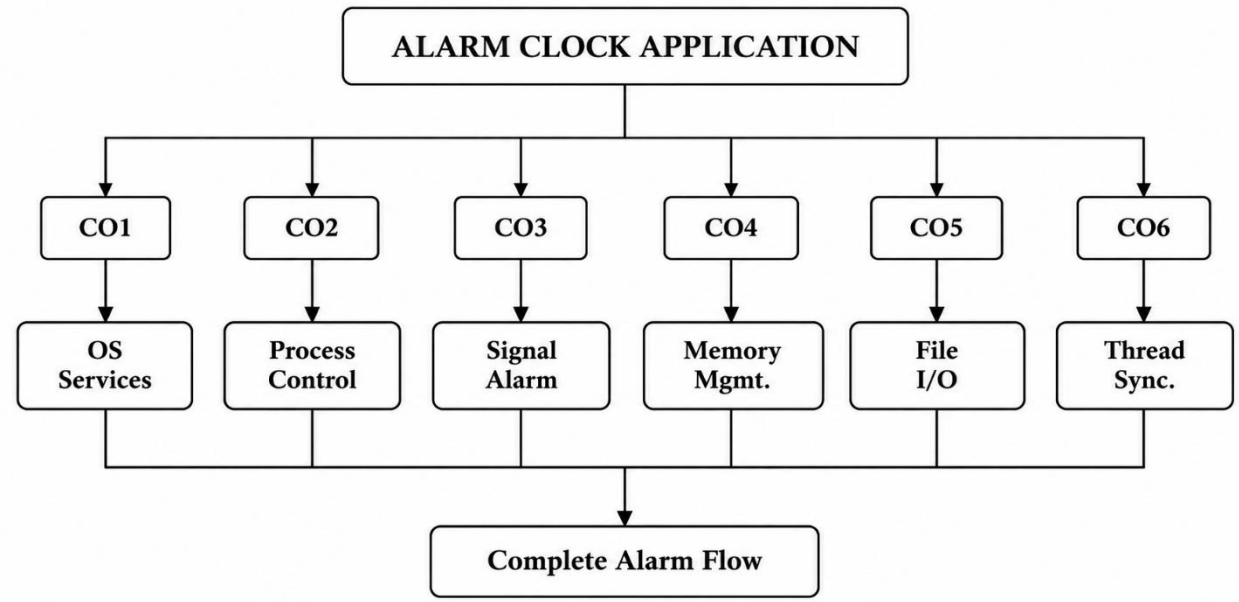
The project uses a mutex to protect shared alarm state. A condition variable is used as part of the thread coordination mechanism. These synchronization mechanisms help coordinate activities that may occur between different execution contexts.

The use of pthread_join() ensures that the application waits for the monitor thread to complete its work before continuing with the corresponding cleanup process.

2.1.7 Integration of Operating System Concepts

The main feature of the project is that CO1 to CO6 are not implemented as completely separate applications. They are integrated into the same Alarm Clock application.

The general relationship between the components can be represented as:



This integration allows the project to demonstrate how multiple operating-system concepts can work together in a single user-space application.

2.2 Existing Methods / Algorithms

A basic alarm program can repeatedly poll the current time in a loop. Polling requires repeated time checks and does not demonstrate asynchronous signal handling.
In a polling-based approach, the application repeatedly obtains the current system time and compares it with the required alarm time. The general process can be represented as:
Start
↓
Read Current Time
↓
Compare With Alarm Time
↓
Alarm Time Reached?
↓ No
Read Current Time Again
↓
Repeat

This approach requires continuous checking of the system time while waiting for the alarm. The program remains occupied with repeated time comparisons until the required time is reached. Another approach is to use a timer facility supplied by the operating system. Instead of continuously checking the time, the application calculates the required delay and asks the operating system to provide a notification after that delay.
The proposed project uses this second approach by calculating a delay and scheduling alarm(seconds).

The timer-based approach used in the project can be represented as:
Start
↓
Obtain Current Time
↓
Read Alarm Time
↓
Calculate Delay
↓
Schedule alarm(seconds)
↓
Wait for SIGALRM
↓
Alarm Event
↓
Trigger Alarm Processing
This approach is directly related to the project's objective of demonstrating Linux timing and signal facilities.

2.2.1 Comparison of Polling and Timer-Based Operation

Feature	**Polling Approach**	**Project Approach**
Time checking	Repeatedly checks time	Calculates required delay
Waiting mechanism	Continuous loop	Operating-system timer
Alarm notification	Time comparison	SIGALRM
Main concept demonstrated	Time polling	Signal and timer handling
Linux signal usage	Not necessarily required	Uses SIGALRM
Project integration	Limited	Integrated with CO1–CO6

The project therefore focuses on demonstrating operating-system-supported alarm scheduling rather than implementing the alarm using only repeated time comparisons.

2.3 Proposed Approach
The proposed approach uses the current system time from time(), converts it with localtime_r(), builds the target time using mktime(), calculates the waiting duration, and schedules the Linux alarm.

The application first obtains the current system time. The time information is converted into a usable local-time representation. The user then provides the required alarm time.
The application determines the target alarm time and calculates the difference between the current time and the requested alarm time. This difference represents the number of seconds for which the application needs to wait.

The calculated duration is then supplied to the Linux alarm() facility.
The basic sequence is:
Obtain Current Time
↓
Convert Current Time
↓
Read User Alarm Time
↓
Build Target Time
↓
Calculate Waiting Duration
↓
Schedule alarm(seconds)
↓
Wait for SIGALRM

When the scheduled time is reached, SIGALRM is generated. The background worker waits for the alarm event and calls the alarm trigger routine when the event occurs.

The trigger routine then demonstrates process control and synchronization before cleaning up the alarm.
The complete proposed approach can be represented as follows:

User Starts Application
↓
Display Menu
↓
Select Set Alarm
↓
Validate Alarm Input
↓
Obtain Current Time
↓
Calculate Alarm Delay
↓
Schedule alarm()
↓
Background Worker Waits
↓
SIGALRM Received
↓
Alarm Trigger Routine
↓
Process Control
↓

Thread Synchronization
↓
File Operations
↓
Memory Release
↓
Return to Menu

2.3.1 Alarm Input and Validation

The user enters the required alarm time through the terminal. The application validates the entered time before scheduling the alarm.

Validation is important because the timer operation depends on receiving a meaningful time value. Invalid input should not result in an incorrectly scheduled alarm.

After successful validation, the application proceeds with the calculation of the waiting duration.

2.3.2 Alarm Delay Calculation

The alarm is scheduled using a delay rather than directly providing a clock time to alarm(). The application obtains the current time and determines the target alarm time. The difference between these two values is used as the delay.

Conceptually:
Waiting Duration =
Target Alarm Time - Current System Time
The resulting duration is passed to alarm().

2.3.3 Signal Notification

After alarm() is scheduled, the application waits for the operating-system notification.

When the specified duration expires, Linux generates SIGALRM. The project uses sigaction() as part of its signal-handling mechanism.

This demonstrates asynchronous event notification because the alarm event is generated by the operating system after the scheduled delay.

2.3.4 Alarm Trigger Processing

After receiving the alarm event, the project performs the alarm-trigger processing.

The trigger stage is connected with other operating-system concepts demonstrated by the project. These include process control through CO2 and thread synchronization through CO6.

The alarm event therefore acts as an important point where several project components become connected.

2.3.5 Resource Cleanup

After the alarm is completed or cancelled, resources associated with the operation are released.

The cleanup process can include:

- Cancelling the timer when required.
- Releasing dynamically allocated memory.
- Completing the monitor thread.
- Closing files.
- Removing the stored alarm information when required.
- Terminating the related child process.
- Releasing other alarm-related resources.

Resource cleanup is included to demonstrate responsible management of operating-system and program resources.

2.4 Advantages of the Proposed Approach

The proposed approach provides several advantages within the scope of the Alarm Clock project.

2.4.1 Uses Linux Timing Facilities

The project uses the Linux-provided alarm() mechanism to schedule the alarm. This allows the application to demonstrate how operating-system timer facilities can be used by a user-space program.

2.4.2 Demonstrates Signal Handling

The use of SIGALRM demonstrates asynchronous notification. The application can wait for the operating-system signal instead of continuously comparing the current time in a polling loop.

2.4.3 Integrates CO1 to CO6

A major advantage of the project is the integration of all six course-outcome components into one application.

The integration includes:
CO1: Operating-system services and system information.
CO2: Process creation and process control.
CO3: Alarm timer and SIGALRM.
CO4: Dynamic memory management.
CO5: File input/output.
CO6: POSIX threads and synchronization.

This makes the Alarm Clock a combined demonstration of several operating-system concepts.

2.4.4 Simple Terminal-Based Interface

The application uses a simple terminal interface. The main operations are:
1. Set Alarm
2. View Alarm

3. Cancel Alarm
4. Exit
The simple menu allows the user to interact with the application without requiring a graphical user interface.

2.4.5 Separation of Responsibilities

The project separates different operating-system concepts into different source files. The CO files contain the corresponding demonstrations, while alarm.c coordinates the alarm-related operations.

This organization makes it easier to understand the relationship between individual operating-system concepts and the complete application.

2.4.6 Demonstrates Resource Management

The project includes cleanup operations for timers, memory, files, processes, and threads. For example, when an alarm is cancelled, the application cancels the timer and releases associated resources. After alarm completion, the application performs the required cleanup before returning to the menu.

2.4.7 Demonstrates Multiple Execution Mechanisms

The project demonstrates multiple execution mechanisms in a single application. These include the normal program flow, the alarm event, a background monitor thread, and alarm-related process control.

This allows the project to demonstrate how different execution activities can be coordinated within a Linux environment.

2.4.8 Suitable for Linux-Based Demonstration

The project is designed for a Linux environment and can be compiled and executed using GCC. The use of Linux/WSL provides an environment in which the required system calls, signals, processes, files, and POSIX threads can be demonstrated.

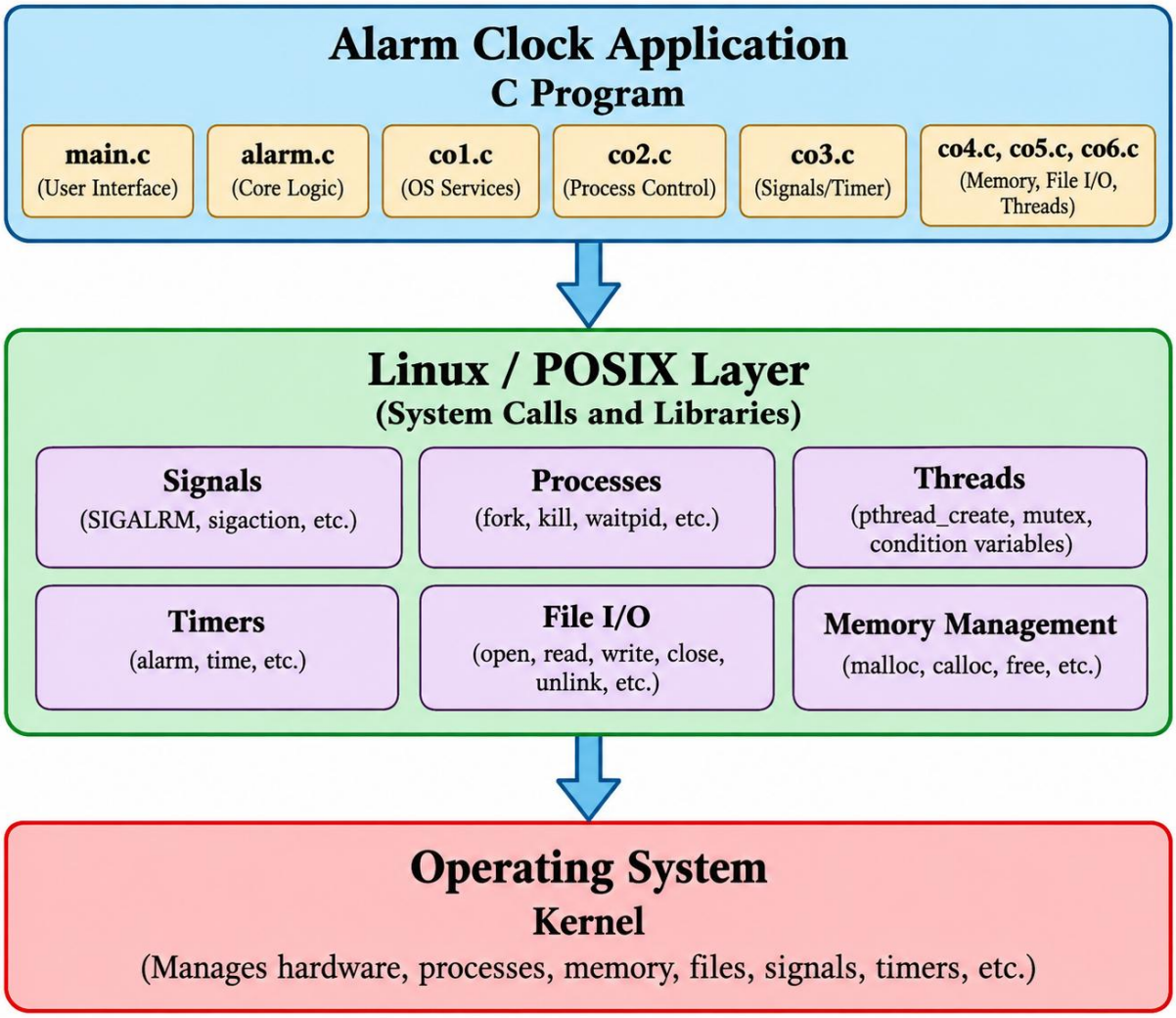
2.5 Background Study of Linux and POSIX Environment

The Alarm Clock project is implemented as a user-space application in a Linux environment. Linux provides the operating-system services required for process management, signals, file operations, timing, and other system-level activities.

POSIX concepts are also relevant to the project, particularly for thread management and synchronization. The use of POSIX threads allows the application to create a monitor thread and coordinate its execution using synchronization mechanisms.

The project therefore provides practical exposure to the relationship between a C application and the operating system.

The overall interaction can be represented as:



The application requests operating-system services through the facilities provided by Linux and POSIX. The operating system then manages the corresponding resources and events.

2.6 Relationship Between the Concepts

The operating-system concepts used in the project are connected through the alarm lifecycle. When the user sets an alarm, the application first obtains the required system and time information. The alarm delay is calculated and the timer is scheduled.

During the waiting stage, the monitor thread coordinates alarm-related activity. When the timer expires, SIGALRM indicates that the scheduled event has occurred.

After the alarm event, process control operations and synchronization mechanisms are demonstrated. File operations are used to manage alarm information, while dynamically allocated memory is released after it is no longer required.

Therefore, the concepts can be viewed as a sequence:
Set Alarm

↓
System Services
↓
Time Calculation
↓
Timer Scheduling
↓
SIGALRM
↓
Process Control
↓
Thread Synchronization
↓
File Operations
↓
Memory Release
↓
Alarm Completion

This relationship shows that the project is not limited to a single operating-system feature. Instead, multiple concepts are combined to implement the complete alarm-management workflow.

2.7 Summary of the Background Study

The background study provides the foundation for understanding the Alarm Clock project. The project uses Linux and POSIX facilities to demonstrate operating-system concepts in a practical application.

The major concepts studied are operating-system services, processes, signals, timers, dynamic memory, file operations, threads, and synchronization.

The project uses time(), localtime_r(), mktime(), and alarm() for alarm scheduling. SIGALRM and sigaction() are used for alarm notification. Process-related operations demonstrate process control, while POSIX threads and synchronization mechanisms demonstrate coordination of shared alarm activity.

The use of file operations and dynamic memory management further demonstrates resource handling within a C application.

Overall, the background study establishes the connection between theoretical operating-system concepts and their practical implementation in the Linux Terminal Alarm Clock project.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The Alarm Clock project is a terminal-based C application and therefore does not require specialized hardware. A standard computer or laptop capable of running Windows with WSL is sufficient for development, compilation, testing, and execution.
The following hardware requirements are considered suitable for the project:

- **Computer or Laptop:** A system capable of running Windows and Windows Subsystem for Linux (WSL).
- **Processor:** A standard modern processor capable of running the Windows host environment and Ubuntu through WSL.
- **RAM:** At least **4 GB RAM** is recommended for the development environment.
- **Storage:** Sufficient storage for the source files, compiler, development tools, Git repository, and project output files.
- **Keyboard:** Required for entering menu selections, alarm time, and other terminal input.
- **Display:** Required for viewing the terminal interface, program output, alarm notifications, and execution results.
-

Since the application operates through the terminal, no special graphical hardware, external alarm device, or dedicated timer hardware is required.
The hardware requirements are therefore relatively simple and suitable for a student development environment.

3.2 Software Requirements

The project requires a Windows host environment together with Ubuntu through WSL for Linux-based compilation and execution. The source code is written in C and compiled using GCC.

Software	Purpose
Windows	Host operating system used for development.
Ubuntu through WSL	Linux environment for compilation and execution.
VS Code	Source-code editing and project organization.
GCC	C compiler and linker.
Git	Version control and project repository management.

3.2.1 Windows

Windows acts as the host operating system for the development setup. The project files can be organized and edited using Windows-based development tools while the actual Linux compilation and execution are performed through WSL.

This arrangement provides a convenient development environment while allowing the project to demonstrate Linux operating-system concepts.

3.2.2 Ubuntu through WSL

Ubuntu is used inside Windows Subsystem for Linux. It provides the Linux environment required for compiling and executing the Alarm Clock application.

The Linux environment is important because the project demonstrates concepts such as signals, processes, timers, file operations, and POSIX threads.

3.2.3 Visual Studio Code

Visual Studio Code is used for writing and organizing the project source code. The project contains multiple C source files and header files, so a source-code editor helps organize the different modules.

The project includes files such as:

- main.c
- alarm.c
- alarm.h
- os_project.h
- co1.c
- co2.c
- co3.c
- co4.c
- co5.c
- co6.c

VS Code is therefore used mainly as the development and editing environment.

3.2.4 GCC

GCC is used to compile the C source files and create the executable Alarm Clock application. The complete project is compiled as one application by passing the required source files to GCC. The -pthread option is included because the project uses POSIX thread functionality.

A representative compilation command is:

```
gcc -Wall -Wextra -pthread src/*.c -o alarm_clock
```

Here, GCC compiles the C source files, performs the required linking, and produces the executable named alarm_clock.

3.2.5 Git

Git is used for version control and project repository management. It allows the project source files and development changes to be tracked.

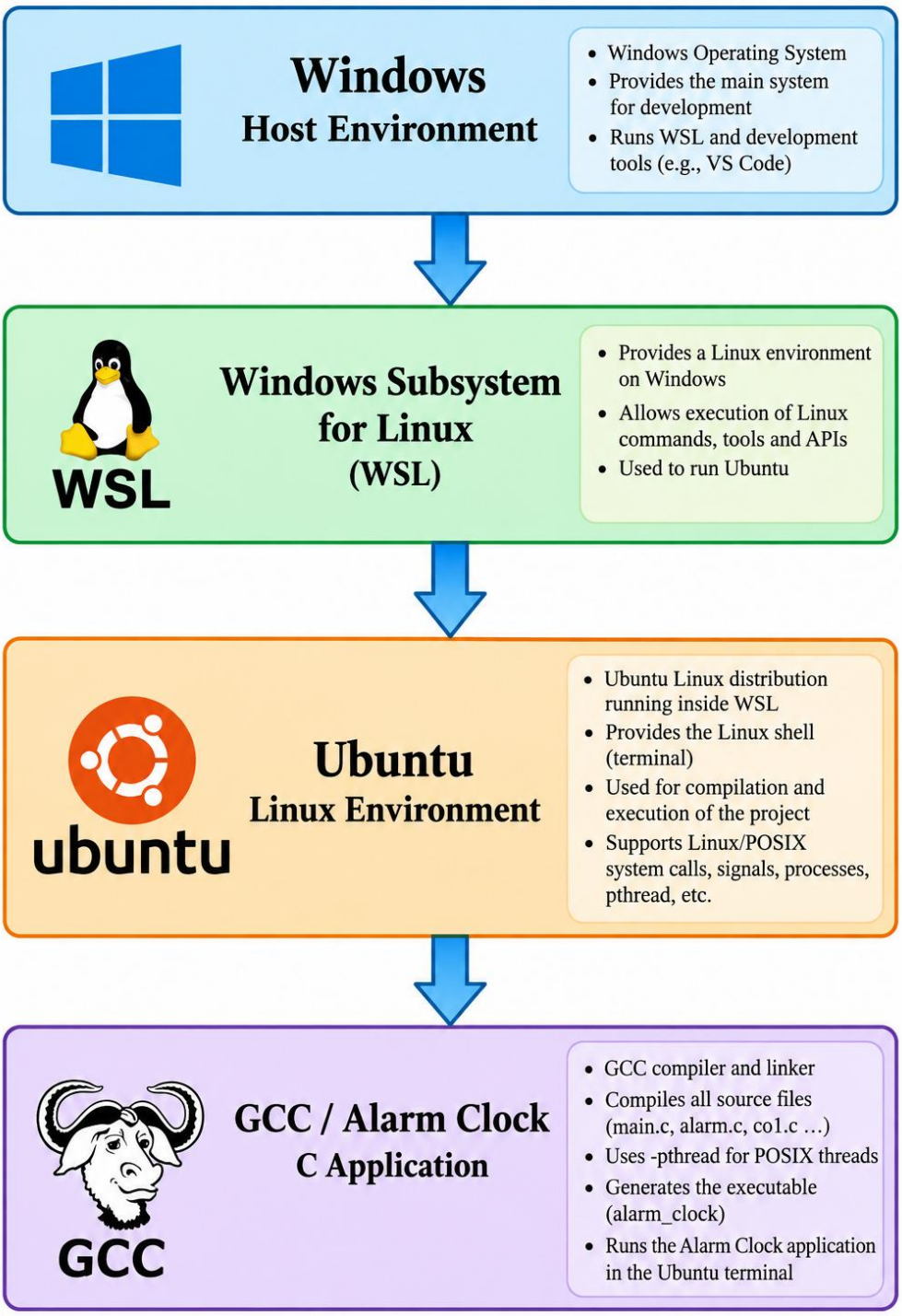
Git is also used to synchronize the local project with the remote project repository. This is useful for maintaining project versions and preserving changes made during development.

3.3 Operating System Environment – Ubuntu / WSL

WSL stands for Windows Subsystem for Linux. It provides a Linux environment on Windows. The project uses Ubuntu inside WSL so that Linux/POSIX system calls, signals, processes and pthread functionality can be compiled and tested in a Linux environment.

The use of WSL is particularly useful for this project because the application is designed around Linux and POSIX operating-system concepts.

The development environment can be represented as:



The source code can be edited using VS Code, while Ubuntu through WSL provides the environment in which the application is compiled and executed.

3.3.1 Role of WSL in the Project

WSL provides the Linux environment required by the project without requiring a separate physical Linux computer.

The project makes use of operating-system concepts that are demonstrated in the Linux environment. These include:

- Linux system services.
- Process creation and control.
- Signals and SIGALRM.
- Alarm timers.
- File input/output.
- Dynamic memory management.
- POSIX threads.

Mutex and condition-variable based synchronization.

The WSL environment therefore connects the development setup with the Linux-specific functionality required by the project.

3.3.2 Terminal-Based Execution

The Alarm Clock application is executed through the Ubuntu terminal. After compilation, the generated executable can be started using:

./alarm_clock

The application then displays its terminal-based menu:

LINUX TERMINAL ALARM CLOCK

1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. Exit

The user interacts with the application through the keyboard. Alarm information and system-level messages are displayed directly in the terminal.

3.3.3 Compilation and Execution Environment

The general development process is:

Write C Source Code
↓
Save Project Files
↓
Open Ubuntu / WSL
↓
Navigate to Project Directory
↓

Compile Using GCC
↓
Generate alarm_clock
↓
Execute Program
↓
Test Alarm Functions
This process provides a straightforward development and testing environment for the project.

3.4 Development Tools and Compilers

The source code is edited using VS Code. Compilation is performed with GCC. The complete project is compiled as one application by passing main.c, alarm.c, and the CO source files to GCC, together with the -pthread option.

3.4.1 Source-Code Editor

VS Code is used to create and modify the C source files. The multi-file structure of the project makes it useful to have an editor that can display and manage multiple source files.
The source files are organized according to their responsibilities. The main application flow is handled through main.c and alarm.c, while the CO files demonstrate the corresponding operating-system concepts.

3.4.2 Compiler

GCC converts the C source files into machine-executable code. Since the project contains multiple source files, GCC compiles the required files and links them into one executable.
The compilation command used for the project is:

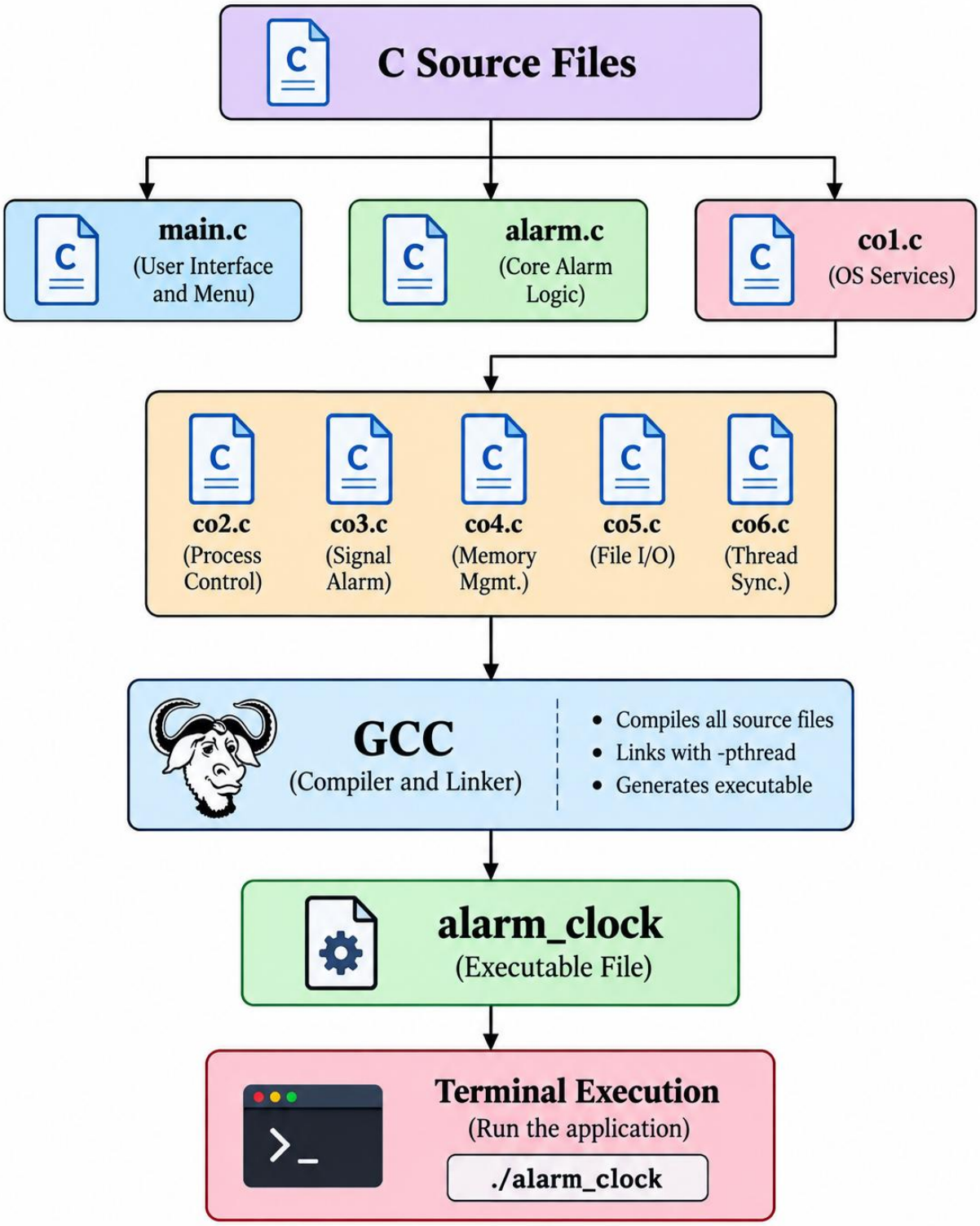
gcc -Wall -Wextra -pthread src/*.c -o alarm_clock

The main parts of this command are:

- **gcc** – invokes the GNU C compiler.
- **-Wall** – enables commonly useful compiler warnings.
- **-Wextra** – enables additional compiler warnings.
- **-pthread** – enables the required POSIX thread compilation and linking support.
- **src/*.c** – includes the C source files in the src directory.
- **-o alarm_clock** – specifies the name of the generated executable.

After successful compilation, the application can be executed using:
./alarm_clock

3.4.3 Project Compilation Structure
The project uses multiple source files but produces a single executable.



This structure allows the different course-outcome components to remain organized while still forming one complete Alarm Clock application.

3.4.4 Development Workflow

The development workflow followed for the project can be summarized as:

Project Planning
↓
Create C Source Files
↓
Implement Alarm Logic
↓
Integrate CO1–CO6
↓
Compile Using GCC
↓
Resolve Compilation Issues
↓
Execute in Ubuntu / WSL
↓
Test Set / View / Cancel / Exit
↓
Record Execution Results
↓
Maintain Project Using Git

This development setup supports both implementation and testing of the Alarm Clock project in a Linux-based environment while retaining Windows as the host system.

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C Programming

The Alarm Clock application is implemented in the **C programming language**. C is used because the project requires direct interaction with Linux and POSIX operating-system facilities such as processes, signals, timers, files, dynamic memory, and threads.

The application is organized into multiple C source files. The main application flow is handled by main.c, while the alarm-related operations are coordinated through alarm.c. The individual course-outcome components are implemented through the CO source files.

The major source files used in the project include:

- main.c – application entry point and menu flow.
- alarm.c – alarm-related coordination and operations.
- alarm.h – alarm-related declarations.
- os_project.h – shared project definitions.
- co1.c – operating-system services.
- co2.c – process control.
- co3.c – signal and alarm functionality.
- co4.c – dynamic memory management.
- co5.c – file input/output.
- co6.c – POSIX thread and synchronization functionality.
-

The use of separate source files allows the different operating-system concepts to be organized according to their responsibilities while still producing one complete executable.

The C language also provides access to the functions and data structures required for Linux systems programming. The project uses standard C functionality together with Linux and POSIX APIs.

The overall relationship between the C application and the operating-system facilities can be represented as:

C Source Code
↓
C Standard / POSIX / Linux APIs
↓
GCC Compilation
↓
Alarm Clock Executable
↓
Linux / Ubuntu Execution

4.2 GCC Compiler

GCC, or GNU Compiler Collection, is used to compile and link the C source files into one executable named alarm_clock.

The project contains multiple source files, so the compiler processes the required C files and combines them during the linking stage. The -pthread option is included because the application uses POSIX thread functionality.

A representative compilation command used for the project is:

gcc -Wall -Wextra -pthread src/*.c -o alarm_clock

The important parts of the command are:

Option / Command	Purpose
gcc	Invokes the GCC compiler.
-Wall	Enables commonly useful compiler warnings.
-Wextra	Enables additional compiler warnings.
-pthread	Provides the required support for POSIX threads.
src/*.c	Includes the C source files in the source directory.
-o alarm_clock	Creates the executable with the name alarm_clock.

The compilation process can be represented as:
main.c
alarm.c
co1.c
co2.c
co3.c
co4.c
co5.c
co6.c
↓
GCC
↓
alarm_clock
↓
Terminal Execution

The use of compiler warning options helps identify potential issues during compilation. After successful compilation, the generated executable is run from the Ubuntu terminal.
The executable can be started using:
./alarm_clock

4.3 Linux System Calls and APIs

The Alarm Clock project uses several Linux and POSIX APIs to demonstrate operating-system concepts. These functions are distributed across the CO modules according to their respective responsibilities.

API / Function	Use in the Project
getpid(), getppid()	Obtain current and parent process IDs.
uname()	Obtain Linux operating-system and kernel information.
write()	Demonstrate direct Linux output through a system call.
fork()	Create the child process used by CO2.
kill()	Send SIGTERM to the child process.
waitpid()	Wait for and collect child-process termination.
alarm()	Schedule the SIGALRM timer.
sigaction()	Install the SIGALRM handler.
sigwait()	Wait for SIGALRM in the alarm worker.
open(), read(), write(), close(), unlink()	Store, load and remove alarm data.
pthread_create(), pthread_join()	Create and join the monitor thread.

4.3.1 Process Identification APIs

The functions getpid() and getppid() are used to obtain process information.
getpid() provides the process ID of the currently running process, while getppid() provides the parent process ID.

These functions are part of the operating-system service demonstration in CO1. The application displays this information during the alarm setup workflow.

This demonstrates that a user-space application can obtain information about its execution context from the operating system.

4.3.2 Operating-System Information

The uname() function is used to obtain information about the Linux operating-system environment and kernel release.

This allows the application to display information about the environment in which the Alarm Clock is executing.

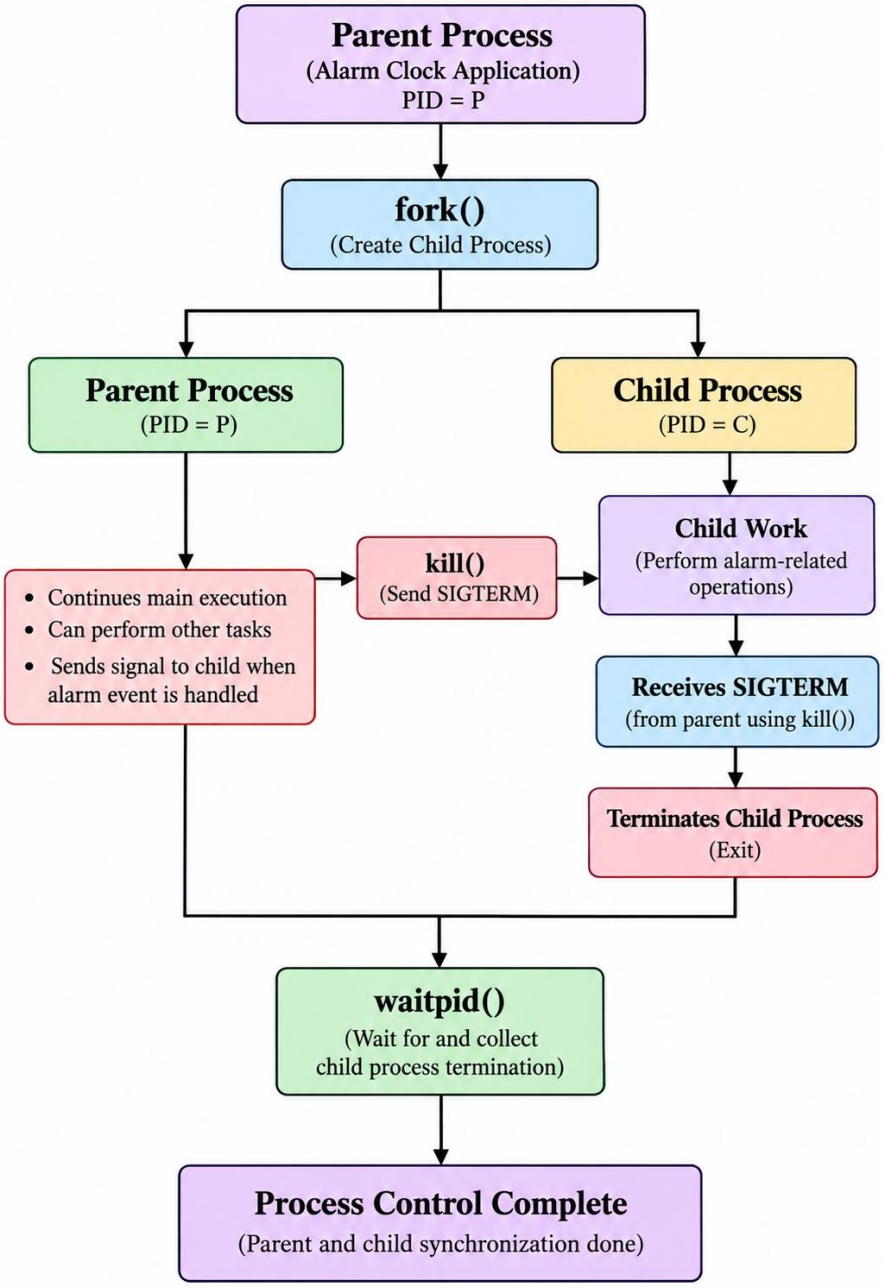
The information is useful for demonstrating the relationship between the application and the underlying Linux environment.

4.3.3 Process Creation and Control

The project uses fork() to demonstrate creation of a child process.

After the child process is created, kill() is used to send SIGTERM to the child process as part of the process-control demonstration. waitpid() is then used to wait for the child-process termination.

The basic sequence is:



This functionality demonstrates parent-child process management in Linux.

4.3.4 Alarm and Signal APIs

The Alarm Clock uses alarm() to schedule the timer.

After the required delay, the operating system generates SIGALRM. The project uses sigaction() as part of its signal-handling mechanism.

The alarm worker also uses sigwait() to wait for the relevant signal event.

The basic relationship is:
Calculate Delay
↓
alarm()
↓
Operating System Timer
↓
SIGALRM
↓
Signal Waiting / Handling
↓
Alarm Trigger
This is one of the main operating-system concepts demonstrated by the project.

4.3.5 File APIs

The project stores alarm-related information in:

data/alarms.txt

File operations are performed using Linux file APIs including:

- open()
- read()
- write()
- close()
- unlink()

These functions allow the application to store, access, close, and remove alarm information as required by the application workflow.

4.3.6 Thread APIs

The project uses POSIX thread functions for the monitor thread.

pthread_create() is used to create the monitor thread, while pthread_join() is used to wait for the thread to complete.

The thread-related functionality is combined with synchronization mechanisms to coordinate shared alarm state.

4.4 POSIX Threads / Synchronization

CO6 uses pthread_create() to create a monitor thread. A mutex protects the shared alarm state, while a condition variable is used to signal the monitor when the alarm event occurs.

pthread_join() is used to wait for the monitor thread to finish.

The thread-based operation can be represented as:

Main Alarm Flow
|
↓
pthread_create()
|
↓
Monitor Thread
|
↓
Wait for Alarm Event
|
↓
Condition Variable
|
↓
Alarm Event Occurs
|
↓
Shared Alarm State
|
↓
Mutex Protection
|
↓
Monitor Completes
|
↓
pthread_join()

4.4.1 Monitor Thread

The monitor thread is responsible for participating in the alarm-event workflow.

Creating a separate thread allows the application to demonstrate concurrent execution within the Alarm Clock program.

The thread works with the alarm state while the main application coordinates the overall alarm operation.

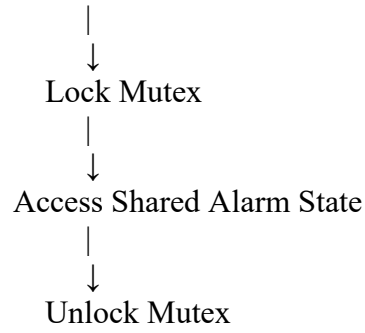
4.4.2 Mutex

A mutex is used to protect the shared alarm state.

Since more than one execution context may access the alarm-related state, synchronization is required to coordinate access.

The basic concept is:

Thread / Execution Context



This demonstrates the use of synchronization to manage shared data.

4.4.3 Condition Variable

A condition variable is used as part of the mechanism for signaling the monitor thread when the alarm event occurs.

The monitor can wait for the required condition instead of continuously checking the shared state.

This provides an example of coordination between different execution activities in a POSIX-based C application.

4.4.4 Thread Completion

After the monitor thread completes its work, pthread_join() is used to wait for the thread.

Joining the thread ensures that the corresponding thread activity has completed before the application proceeds with the related cleanup process.

4.5 Linux Commands and Utilities

Several Linux commands are used during the development, compilation, execution, and testing of the project.

4.5.1 wsl

The wsl command is used from Windows to start the default WSL Linux environment.
wsl
This provides access to the Ubuntu environment used for compiling and executing the project.

4.5.2 cd

The cd command is used to move to the project directory.
For example:

cd /mnt/c/MONISHA/KLH/TriSem/VSCode/src/OSSP_Project/Alarm-Clock

This allows the developer to work directly from the project directory.

4.5.3 ls

The ls command is used to list files and directories.

ls

It is useful for checking the project structure and confirming that the source and data directories are available.

4.5.4 gcc

The gcc command compiles and links the C source files.

gcc -Wall -Wextra -pthread src/*.c -o alarm_clock

The command generates the alarm_clock executable when compilation is successful.

4.5.5 ./alarm_clock

The generated application is executed using:

./alarm_clock

This starts the terminal-based Alarm Clock application.
The user can then interact with the available operations:

LINUX TERMINAL ALARM CLOCK

1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. Exit

4.5.6 Relationship Between Commands

The main commands used during project execution form a simple workflow:

wsl
↓
cd Project Directory
↓
ls
↓
gcc -Wall -Wextra -pthread src/*.c -o alarm_clock
↓
./alarm_clock

This sequence represents the transition from entering the Linux environment to compiling and executing the project.

4.6 Other Tools / Libraries Used

The project uses standard C headers together with POSIX/Linux headers such as pthread.h, signal.h, unistd.h, sys/wait.h, fcntl.h, and sys/utsname.h as required by the individual modules.

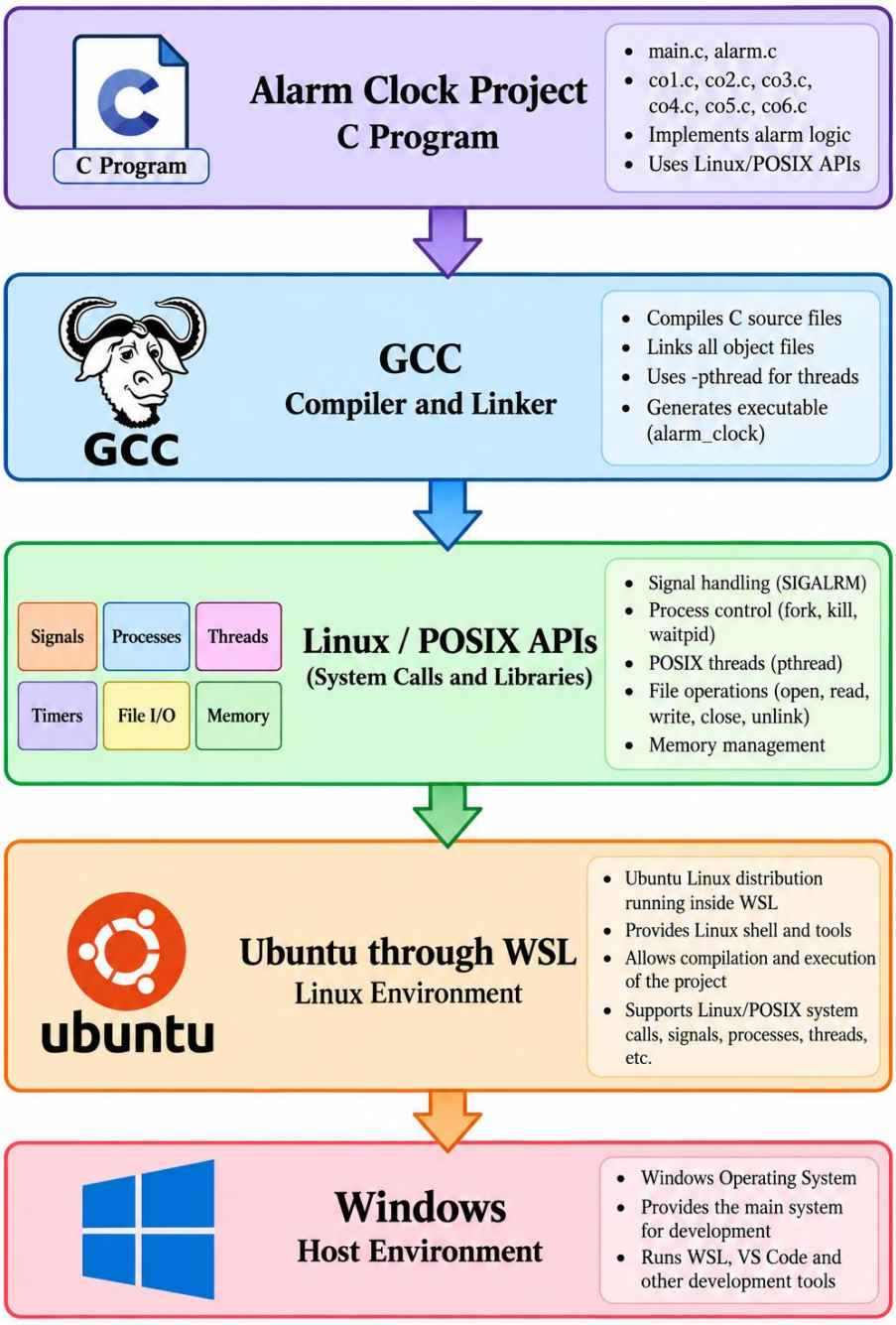
These headers provide declarations and definitions needed to use the corresponding operating-system facilities.

Header	Related Functionality
pthread.h	POSIX threads and synchronization.
signal.h	Signals and signal-related operations.
unistd.h	Linux/POSIX functions such as process and alarm-related operations.
sys/wait.h	Waiting for child processes using process-control functions.
fcntl.h	File-opening and file-control operations.
sys/utsname.h	Operating-system and kernel information through uname().

The use of these headers allows the project source files to access the Linux and POSIX facilities required by the individual course-outcome components.

4.6.1 Overall Technology Stack

The technologies and tools used in the project can be summarized as:



4.6.2 Summary

The technology stack provides all the components required to develop and execute the Alarm Clock application. **C** is used for implementation, **GCC** is used for compilation and linking, **Ubuntu through WSL** provides the Linux execution environment, and **Git** supports version control.

The Linux and POSIX APIs provide the operating-system functionality demonstrated by CO1 to CO6. Signals and timers support alarm scheduling, process APIs demonstrate process control, file APIs manage alarm data, dynamic memory supports alarm objects, and POSIX threads provide monitor-thread synchronization.

Together, these technologies form the development and execution environment for the Linux Terminal Alarm Clock project.

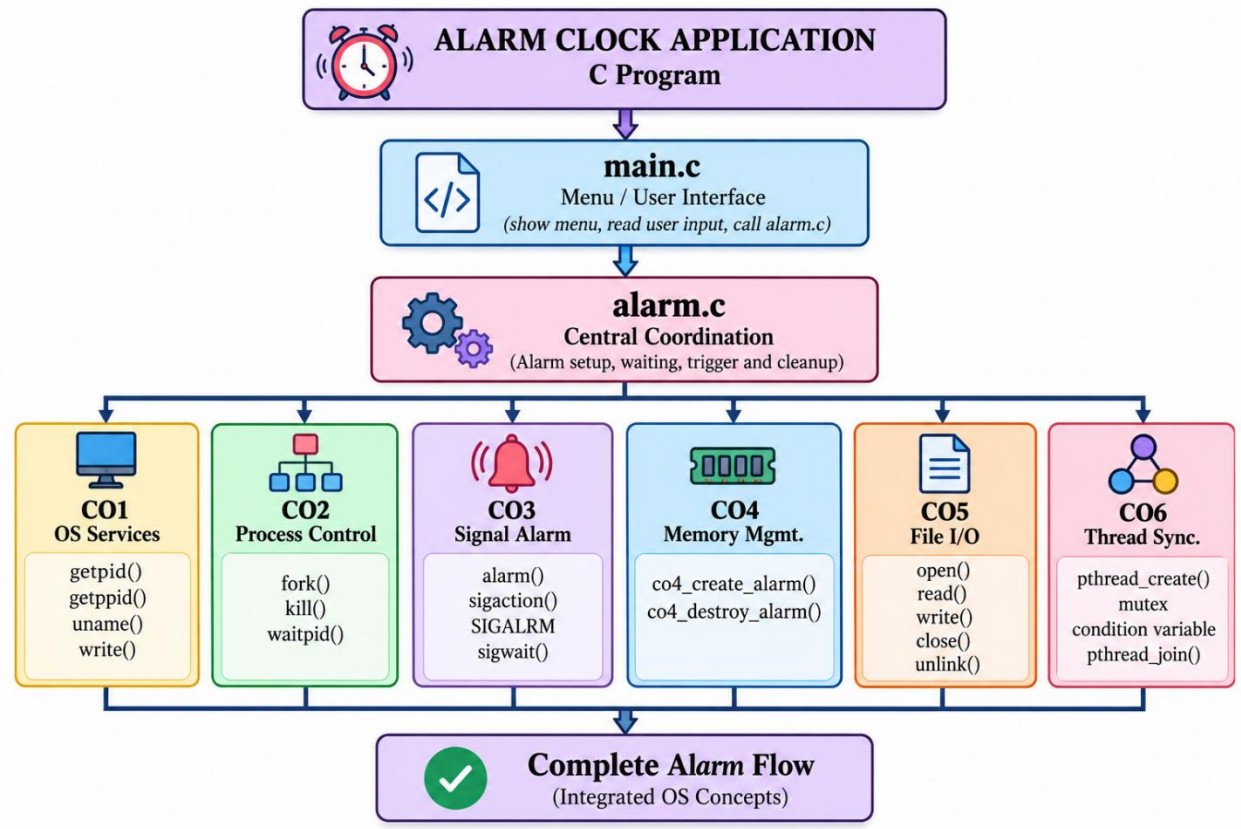
CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Alarm Clock application is designed as a single C-based terminal application. The application has one main entry point in main.c. The menu functions call the functions provided by alarm.c. The alarm.c module acts as the central coordinator and connects the CO1–CO6 modules at different stages of the alarm workflow.
The CO modules are not separate programs. Each module implements a particular systems-programming concept, and its functions are called when that concept is required during the execution of the Alarm Clock application.
The overall module structure is shown below.

Module	Responsibility	Main Functions / APIs
main.c	Menu and user interaction	show_menu(), alarm_init(), alarm_set(), alarm_view(), alarm_delete(), alarm_shutdown()
alarm.c	Central coordination	Alarm setup, waiting, trigger and cleanup
co1.c	OS services	getpid(), getppid(), uname(), write()
co2.c	Process control	fork(), kill(), waitpid()
co3.c	Alarm and signals	alarm(), sigaction(), SIGALRM, sigwait()
co4.c	Dynamic memory	co4_create_alarm(), co4_destroy_alarm()
co5.c	File I/O	open(), read(), write(), close(), unlink()
co6.c	Threads and synchronization	pthread_create(), mutex, condition variable, pthread_join()

The architecture separates the different responsibilities while allowing the modules to work together as one application.
The relationship between the modules can be represented as:



This architecture allows the user to interact with a simple menu while the internal modules demonstrate operating-system concepts.

5.2 System Workflow

At startup, main.c initializes the Alarm Clock. The application then displays the current time and the available menu operations.

The user can select one of the following operations:
1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. Exit

When the user selects **Set Alarm**, the application passes control to the alarm module. alarm.c first cancels any previous waiting operation when required. It then runs the CO1 operating-system service layer.

The application reads the requested hour, minute, second, and label. The entered time is validated before continuing with the alarm setup.

After validation, an Alarm object is created dynamically. The target time and waiting duration are calculated. CO3 is then used to schedule the Linux alarm timer.

At this stage, the CO6 monitor thread is started and the alarm information is stored through the CO5 file module. A background alarm waiter is also created.

The overall Set Alarm workflow is:

User Selects Set Alarm
↓
Cancel Previous Waiting Operation
↓
Run CO1 OS Services
↓
Read Alarm Time and Label
↓
Validate Input
↓
Create Alarm Object
↓
Calculate Target Time
↓
Calculate Waiting Duration
↓
Schedule alarm()
↓
Start CO6 Monitor Thread
↓
Save Alarm Using CO5
↓
Background Alarm Waiter
↓
Wait for SIGALRM

When the timer expires, CO3 reports the SIGALRM event. The alarm worker then calls alarm_trigger().

The trigger operation coordinates the remaining modules. The CO6 monitor is notified, the CO2 child process is started, the child process is controlled using SIGTERM, and the parent waits for its termination using waitpid().

After process control is completed, the monitor thread is joined. The stored alarm information is removed through CO5 and the dynamically allocated alarm memory is released through CO4. The alarm-trigger workflow is:

SIGALRM Received
↓
alarm_trigger()
↓
Notify CO6 Monitor
↓
Start CO2 Child Process
↓
Child Performs Alarm Operation
↓
Send SIGTERM
↓
waitpid()

↓
Join Monitor Thread
↓
Remove Alarm File
↓
Release Alarm Memory
↓
Alarm Processing Complete
↓
Return to Menu
This workflow demonstrates how the different course-outcome components participate in one complete alarm operation.

5.3 Flowchart / Data Flow

The Alarm Clock system flow starts when the application is launched and continues until the user exits the program.

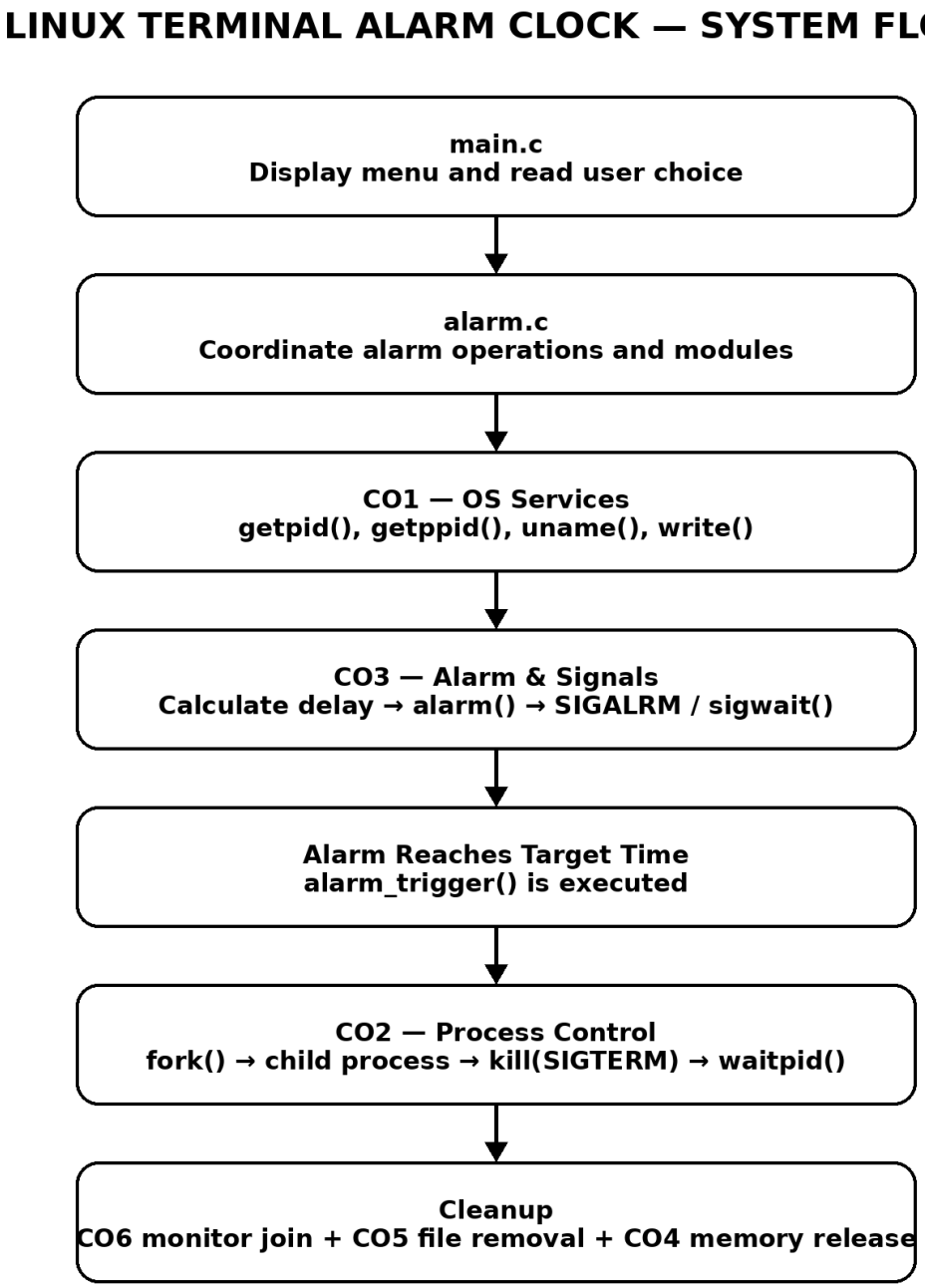


Figure 5.1: Alarm Clock system flow from menu selection to alarm cleanup.

5.4 Module Description

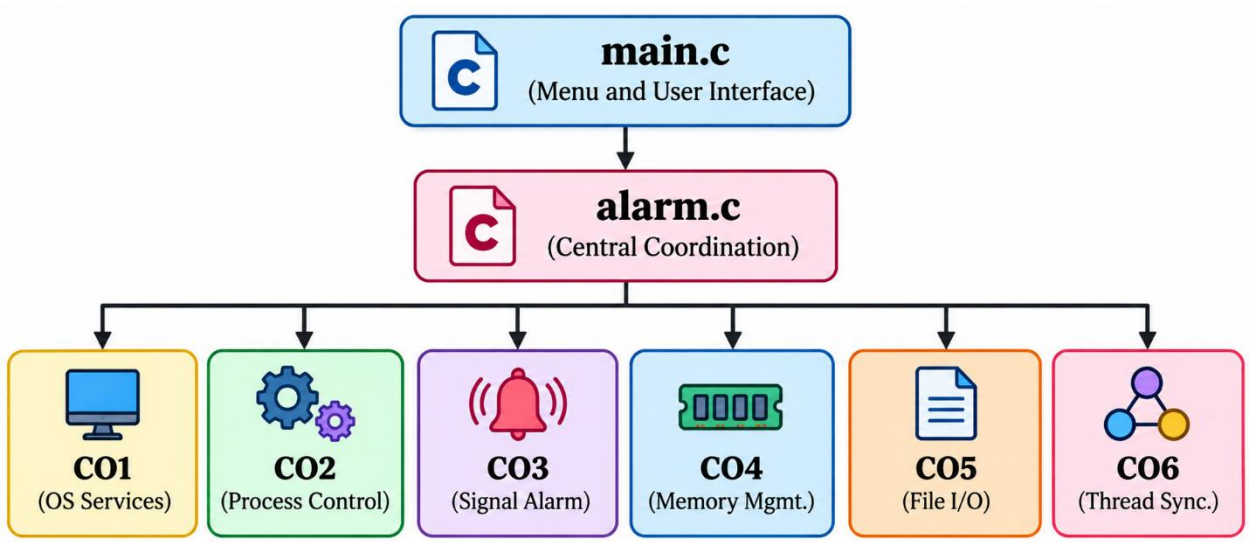
main.c contains the application entry point and menu. It controls the interaction between the user and the Alarm Clock application.

alarm.c contains the central alarm operations. It coordinates alarm setup, timer scheduling, waiting, alarm triggering, and resource cleanup.

alarm.h defines the Alarm structure and the public alarm functions used by the application. The structure provides the data required to maintain the alarm, while the public functions provide the interface between the main program and the alarm module.

os_project.h declares the CO1–CO6 integration functions. The header allows the different source modules to share the required function declarations.

Each CO source file implements its corresponding systems-programming concept.
The architecture therefore follows a modular design:



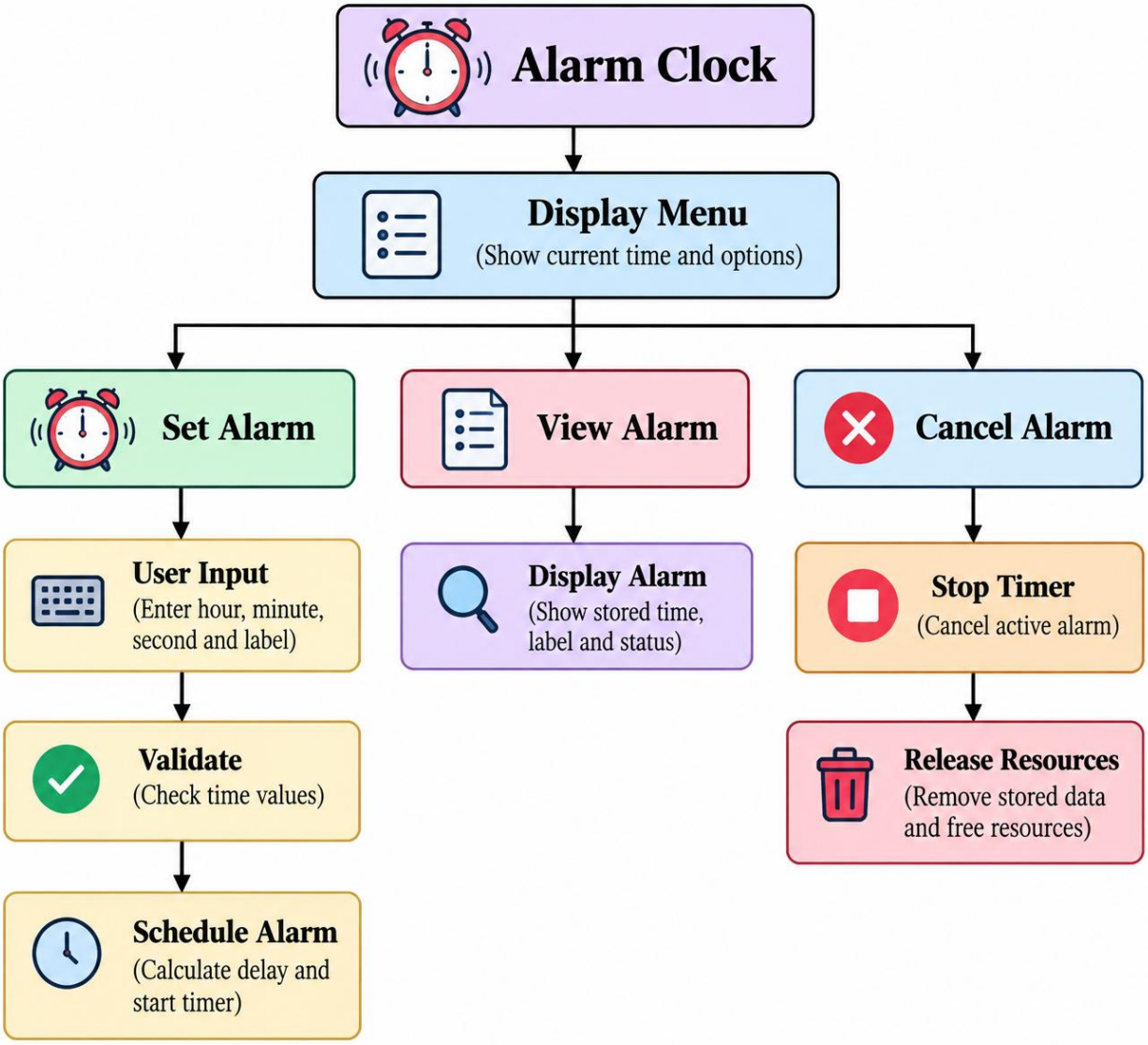
This modular organization makes it possible to identify which source file demonstrates each operating-system concept.

5.5 Input and Output Design

The Alarm Clock application uses a simple terminal-based input and output design.

The **Set Alarm** operation accepts the hour, minute, second, and a label. The entered values are validated before the alarm is scheduled.

The **View Alarm** operation displays the stored time, label, and status of the current alarm.
The **Cancel Alarm** operation stops the timer and removes the stored alarm data. The resources associated with the active alarm are also released as required.
The **Exit** operation releases active resources and terminates the program.
The basic interaction is:



The terminal-based design keeps the user interface simple while the underlying modules demonstrate the required operating-system concepts.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

6.1.1 main.c

main.c initializes the system using alarm_init(), displays the current time and menu, reads the user's choice, and dispatches the selected operation through a switch statement.

It does not directly implement CO1–CO6. Instead, it calls the alarm module, which coordinates the required modules.

The general structure is:

```
main()
  ↓
alarm_init()
  ↓
Display Current Time
  ↓
Display Menu
  ↓
Read User Choice
  ↓
Switch Statement
  ↓
Set / View / Cancel / Exit
```

When the user selects Set Alarm, alarm_set() is called. When View Alarm is selected, alarm_view() is called. Cancel Alarm calls alarm_delete(), while Exit calls alarm_shutdown(). This keeps the main function focused on application control and user interaction.

6.1.2 alarm.c

alarm.c is the integration layer of the project.

The alarm_set() function validates the time, creates the Alarm structure, calculates the target time, schedules CO3, starts the CO6 monitor, saves the alarm using CO5, and creates a background waiter.

The alarm trigger function, alarm_trigger(), runs when the alarm event is received. It coordinates CO2, CO6, CO5, and CO4 during alarm completion and cleanup.

The basic relationship is:

```
alarm_set()
    |
    +---- CO1
    |
```

+---- CO4
|
+---- CO3
|
+---- CO6
|
+---- CO5
|
↓
Wait for Alarm
↓
alarm_trigger()
|
+---- CO2
|
+---- CO6
|
+---- CO5
|
+---- CO4

This makes alarm.c the central integration point of the project.

6.1.3 CO1 – co1.c

The actual application function co1_alarm_service() implements the operating-system service demonstration.

It prints the current process ID and parent process ID using getpid() and getppid().

The module also retrieves Linux and kernel information using uname().

The write() system call is used to demonstrate direct Linux output through a system-level operation.

The CO1 execution can be represented as:

CO1 Alarm Service
↓
getpid()
↓
getppid()
↓
uname()
↓
write()
↓
OS Service Information Displayed

This allows the user to observe information about the process and the Linux environment in which the application is running.

6.1.4 CO2 – co2.c

The CO2 module demonstrates process creation and process control.

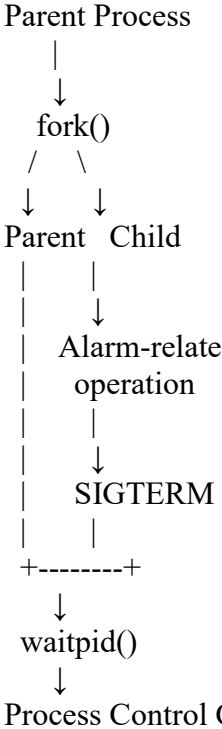
co2_start_alarm_process() calls fork() to create the child process.

The child executes a shell command that produces terminal bell notifications. The parent receives the child PID.

When the child process needs to be stopped, co2_stop_alarm_process() sends SIGTERM using kill().

The parent then waits for the child using waitpid().

The implementation can be represented as:



This demonstrates the creation, control, and termination of a child process.

6.1.5 CO3 – co3.c

CO3 implements the alarm and signal functionality.

co3_schedule_alarm() installs the SIGALRM handler using sigaction(). It blocks SIGALRM for the worker wait set and then calls alarm(seconds) to schedule the timer.

co3_wait_for_alarm() waits for the signal using sigwait().

When the alarm is no longer required, co3_cancel_alarm() calls:

alarm(0)

to cancel the timer.

The CO3 flow is:

Calculate Delay
↓
alarm(seconds)
↓
Wait
↓
SIGALRM
↓
sigwait()
↓
Alarm Event
↓
alarm_trigger()
CO3 therefore provides the timer and signal component of the Alarm Clock.

6.1.6 CO4 – co4.c

CO4 implements dynamic memory management for the Alarm structure.

The Alarm structure is created dynamically when required. The main functions are:

co4_create_alarm()
co4_destroy_alarm()

The memory lifecycle is:
Set Alarm
↓
Create Alarm Structure
↓
Store Alarm Information
↓
Alarm Remains Active
↓
Completion / Cancellation
↓
Destroy Alarm Structure
↓
Memory Released
The dynamically allocated memory is therefore released after it is no longer required.

6.1.7 CO5 – co5.c

CO5 implements Linux file operations for storing alarm information.

The alarm is stored in:

data/alarms.txt

The module provides functions to save, load, and delete the alarm.
The implementation uses:
open()
read()
write()
close()
unlink()
The basic file-operation flow is:
Alarm Information
↓
open()
↓
write()
↓
close()
↓
data/alarms.txt
When the alarm information is no longer required, the file can be removed using unlink().

This module demonstrates how the Alarm Clock uses Linux file operations as part of its overall workflow.

6.1.8 CO6 – co6.c

CO6 creates a POSIX monitor thread and protects the shared Alarm structure using a mutex.

A condition variable allows the monitor to wait for an alarm event. When the alarm event occurs, the monitor is notified and can access the shared alarm state under mutex protection.

The monitor is joined when alarm handling is complete.

The implementation can be represented as:

Create Monitor Thread
↓
pthread_create()
↓
Monitor Waits
↓

Condition Variable
↓
Alarm Event
↓
pthread_cond_signal()
↓
Mutex Protected State
↓
Monitor Completes
↓
pthread_join()
This provides the thread synchronization component of the project.

6.2 Algorithm / Pseudocode

1. Start the Alarm Clock application.
2. Initialize the alarm structure and load stored alarm information if available.
3. Display the current time and menu.
4. If Set Alarm is selected:
    a. Read hour, minute, second and label.
    b. Validate the time values.
    c. Calculate the target time.
    d. Calculate the waiting duration.
    e. Schedule alarm(seconds).
    f. Start the CO6 monitor thread.
    g. Save the alarm to the data file.
    h. Start the background alarm waiter.
5. If View Alarm is selected:
    a. Display the stored time, label and status.
6. If Cancel Alarm is selected:
    a. Cancel the active timer.
    b. Remove stored alarm data.
    c. Release required resources.
7. When SIGALRM is received:
    a. Trigger the alarm event.
    b. Notify the CO6 monitor.
    c. Start the CO2 child process.
    d. Send SIGTERM to the child.
    e. Wait using waitpid().
    f. Join the monitor thread.
    g. Remove the alarm file.
    h. Free the Alarm structure.
8. Return to the menu.
9. If Exit is selected:
    a. Release active resources.
    b. Terminate the program.

6.3 Core C Implementation

The central data structure is Alarm. It stores the hour, minute, second, a label of up to 63 characters plus the terminating null character, and an enabled flag.

The public interface for the alarm functionality is declared in alarm.h, while the CO functions are declared in os_project.h.

The structure provides the common alarm state that is accessed by the alarm-management functions and the CO6 monitor.

Conceptually, the structure contains:

Alarm
- hour
- minute
- second
- label
- enabled

The alarm object is created when an alarm is set and destroyed when the alarm is cancelled or completed.

6.4 Linux System Calls / APIs Used

The implementation uses Linux/POSIX APIs at the points where the corresponding operating-system concept is demonstrated.

The major APIs are:

getpid() / getppid()
↓
Process Information

uname()
↓
Linux / Kernel Information

write()
↓
System-Level Output

fork()
↓
Child Process

kill()
↓
SIGTERM

waitpid()
↓
Child Process Completion

alarm()
↓
Timer

sigaction()
↓
SIGALRM Handling

sigwait()
↓
Signal Waiting

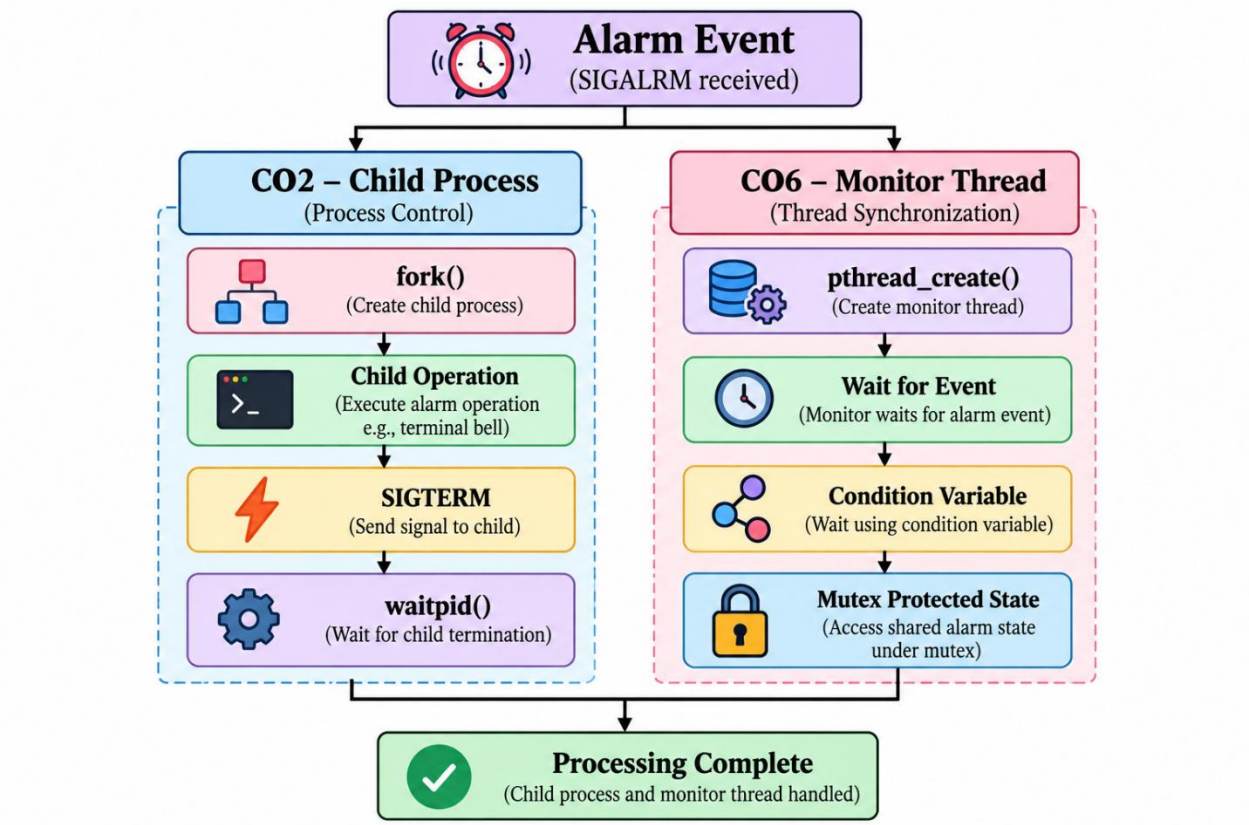
open() / read() / write() / close() / unlink()
↓
File Operations

pthread_create() / pthread_join()
↓
Thread Management
These functions connect the C implementation with the Linux/POSIX operating-system facilities demonstrated by the project.

6.5 Process / Thread Implementation

The alarm trigger starts a child process through CO2. The child produces terminal alarm notifications. The parent controls the child with SIGTERM and waits for termination. Separately, CO6 maintains a monitor thread that waits on a condition variable and accesses the shared alarm state under a mutex.

The two execution mechanisms operate as part of the same alarm workflow:



This implementation demonstrates both process control and POSIX thread synchronization within the same application.

6.6 IPC / Synchronization Implementation

Signal delivery provides asynchronous notification for the alarm.

CO6 uses pthread_cond_signal() to notify the monitor thread when the alarm event occurs. The shared alarm state is protected using pthread_mutex_lock() and pthread_mutex_unlock().

The synchronization sequence is:
Alarm Event
↓
pthread_cond_signal()
↓
Monitor Thread Wakes
↓
pthread_mutex_lock()
↓
Access Shared Alarm State
↓
Process Alarm State
↓

pthread_mutex_unlock()
↓
Monitor Completion

The process-control part uses signals to control the child process, while the thread component uses mutexes and condition variables to coordinate access to the shared alarm state.

6.7 Memory Management Implementation

CO4 allocates an Alarm structure when required and releases it using free().

The alarm object is created when the user sets an alarm. The object remains available while the alarm is active.

During cancellation, replacement, completion, or shutdown, alarm.c destroys the current alarm object so that the allocated memory is released.

The memory-management lifecycle is:

Alarm Requested
↓
Allocate Alarm Structure
↓
Store Time / Label / Status
↓
Alarm Active
↓
Completion / Cancellation / Replacement
↓
co4_destroy_alarm()
↓
free()
↓
Memory Released

This implementation demonstrates dynamic memory allocation and deallocation as part of the complete Alarm Clock workflow.

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled in Ubuntu/WSL using GCC. The complete build command from the project README is:
gcc -Wall -Wextra -std=c11 -pthread src/main.c src/alarm.c src/co1.c src/co2.c src/co3.c src/co4.c src/co5.c src/co6.c -o alarm_clock
The command compiles all source files and links them into one executable named alarm_clock. The -pthread option enables POSIX thread support.

7.2 Execution Commands

Command	Purpose
wsl	Start the default WSL Linux environment from Windows.
cd <project-folder>	Move to the Alarm Clock project directory.
ls	List project files and folders.
gcc ... -o alarm_clock	Compile and link the project.
./alarm_clock	Run the compiled Alarm Clock executable.

7.3 Sample Inputs and Outputs

The application starts with the Linux Terminal Alarm Clock menu. The user can choose Set Alarm, View Alarm, Cancel Alarm or Exit. During Set Alarm, the program asks for hour, minute, second and a label.

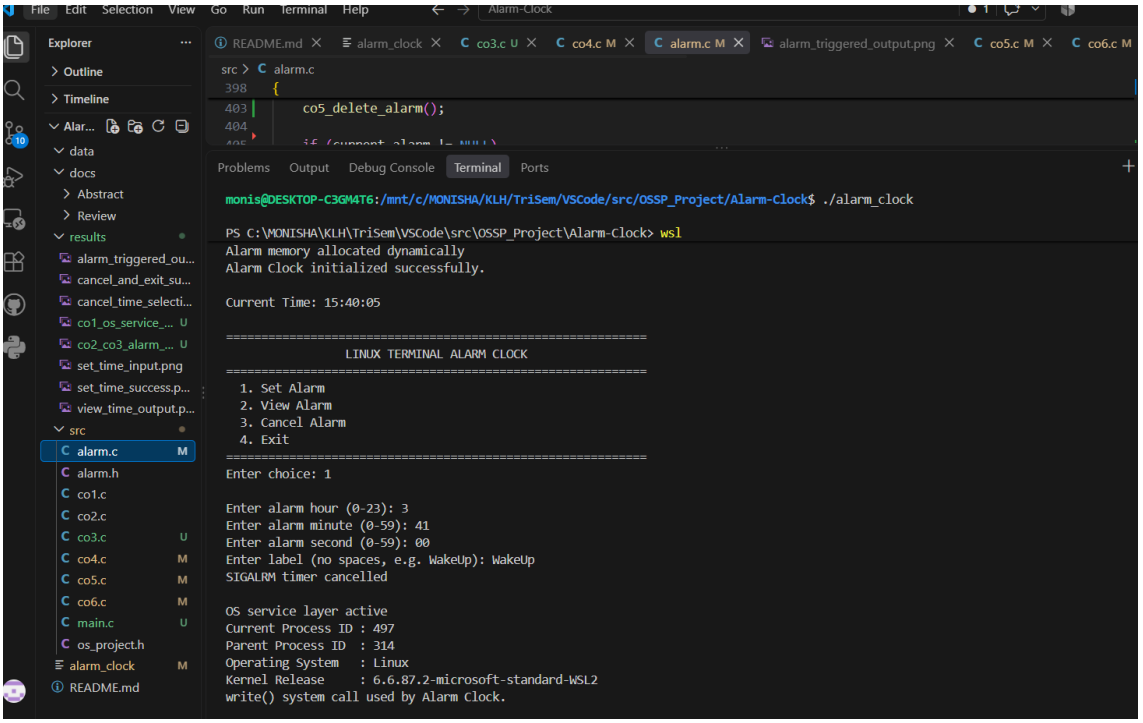


Figure 7.1: Project startup and terminal menu.

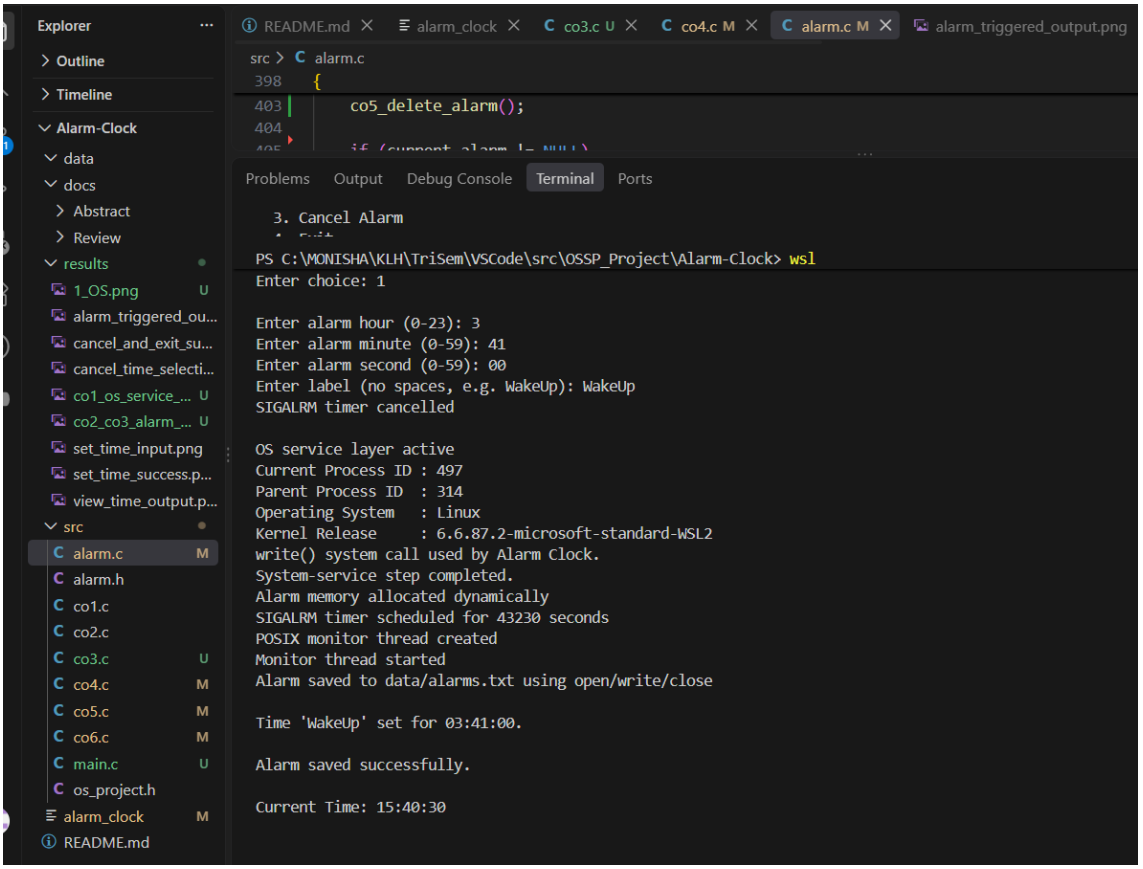


Figure 7.2: Alarm time and label input.

7.4 Screenshots of Execution

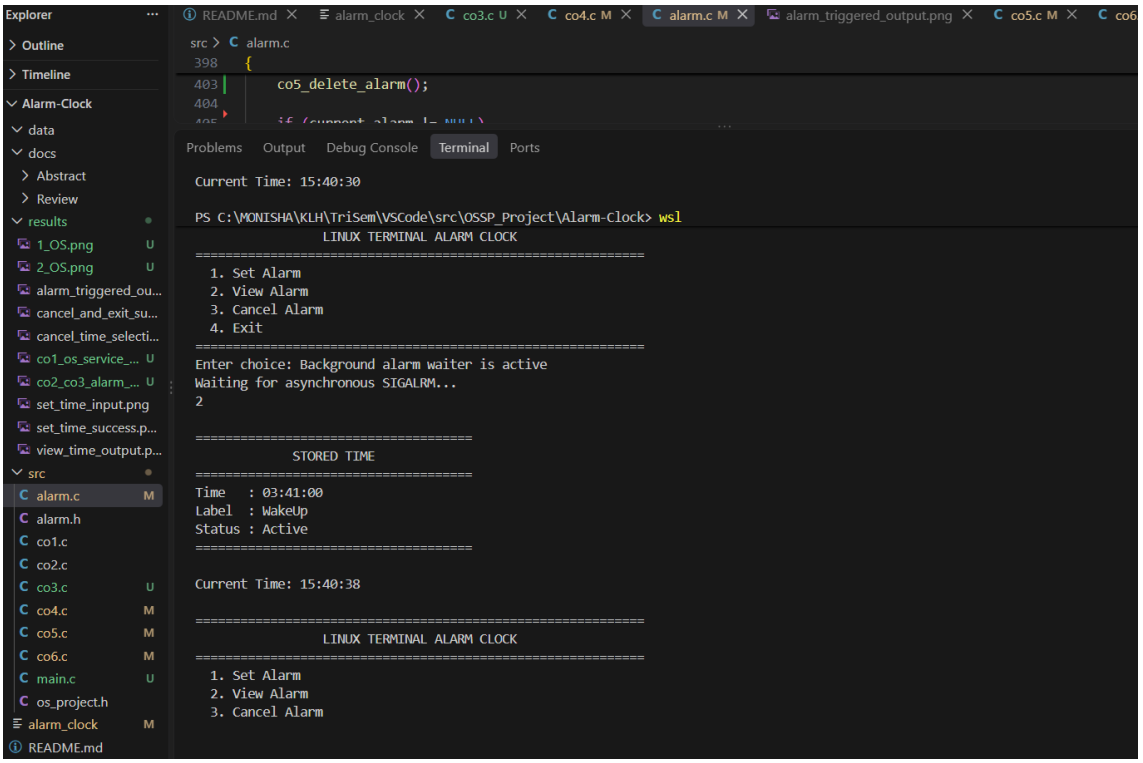


Figure 7.3: CO1 OS service output showing process and Linux information.

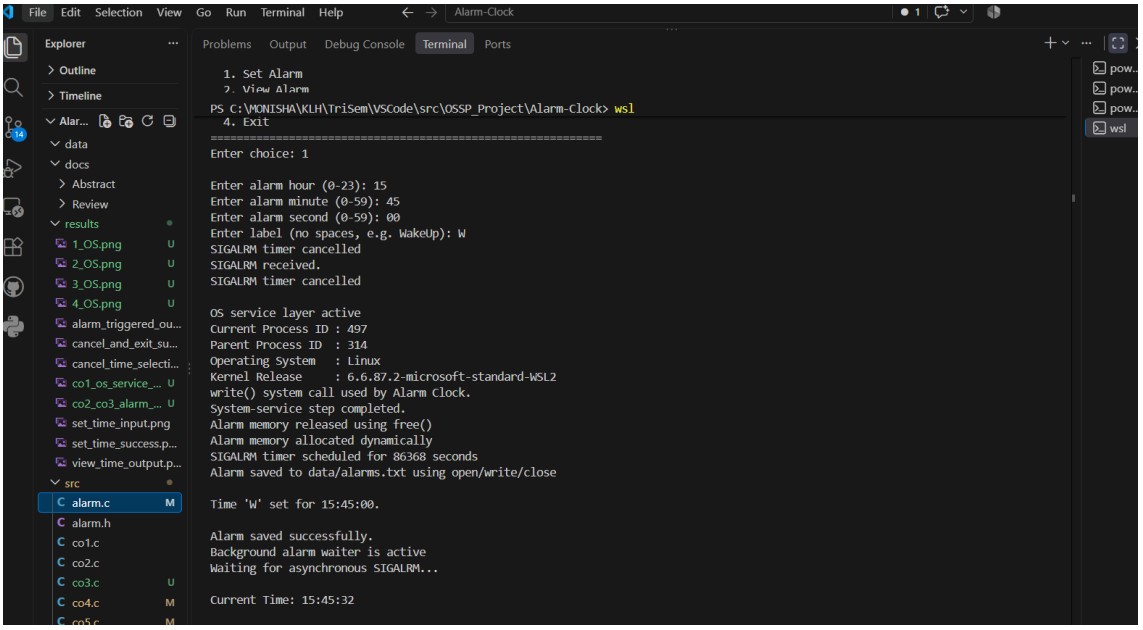


Figure 7.4: CO3 SIGALRM timer and signal-related output.

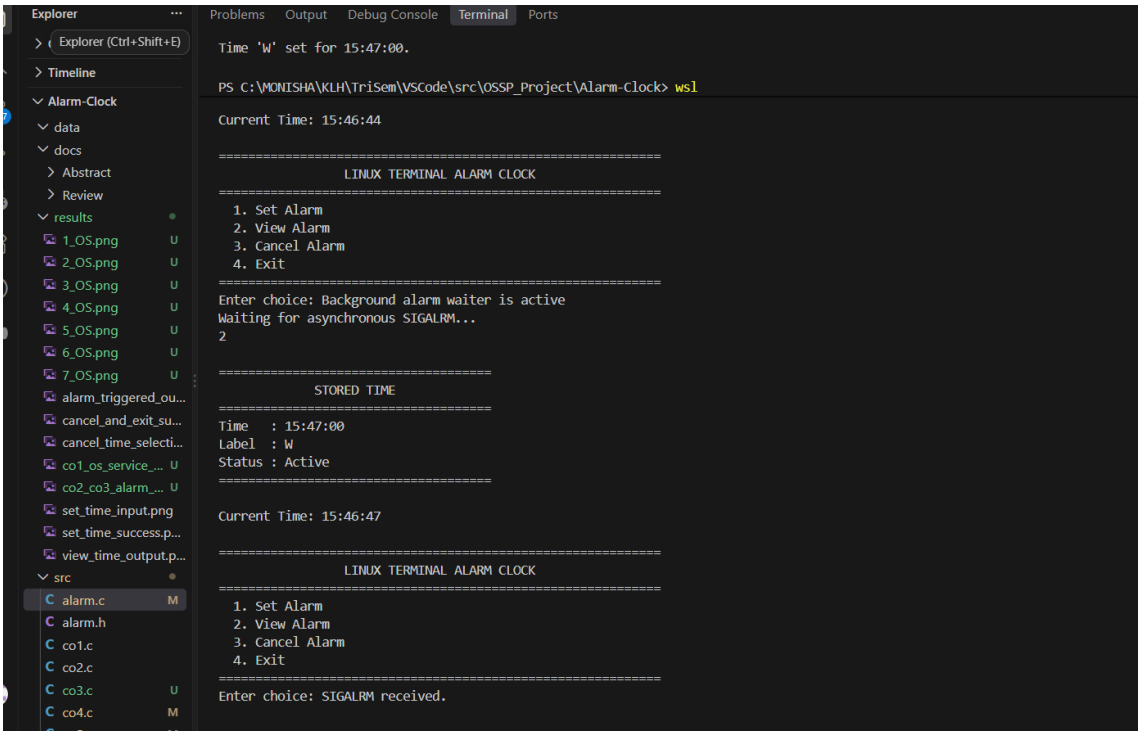


Figure 7.5: View Alarm output showing stored time, label and status.

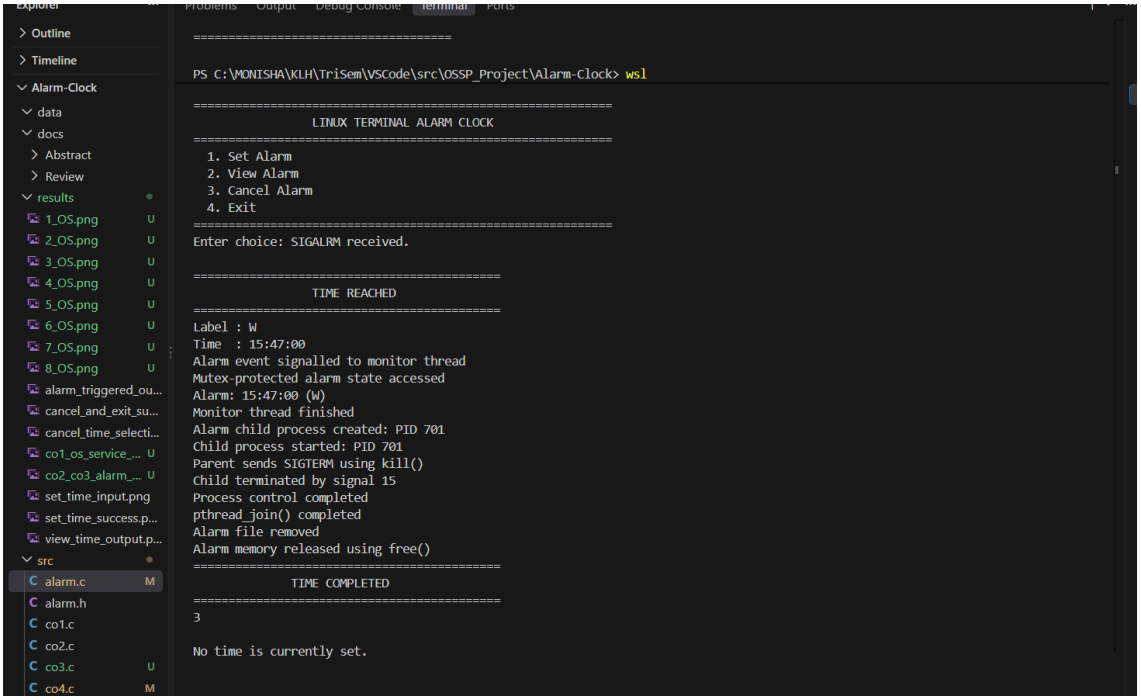


Figure 7.6: CO2 process-control output.

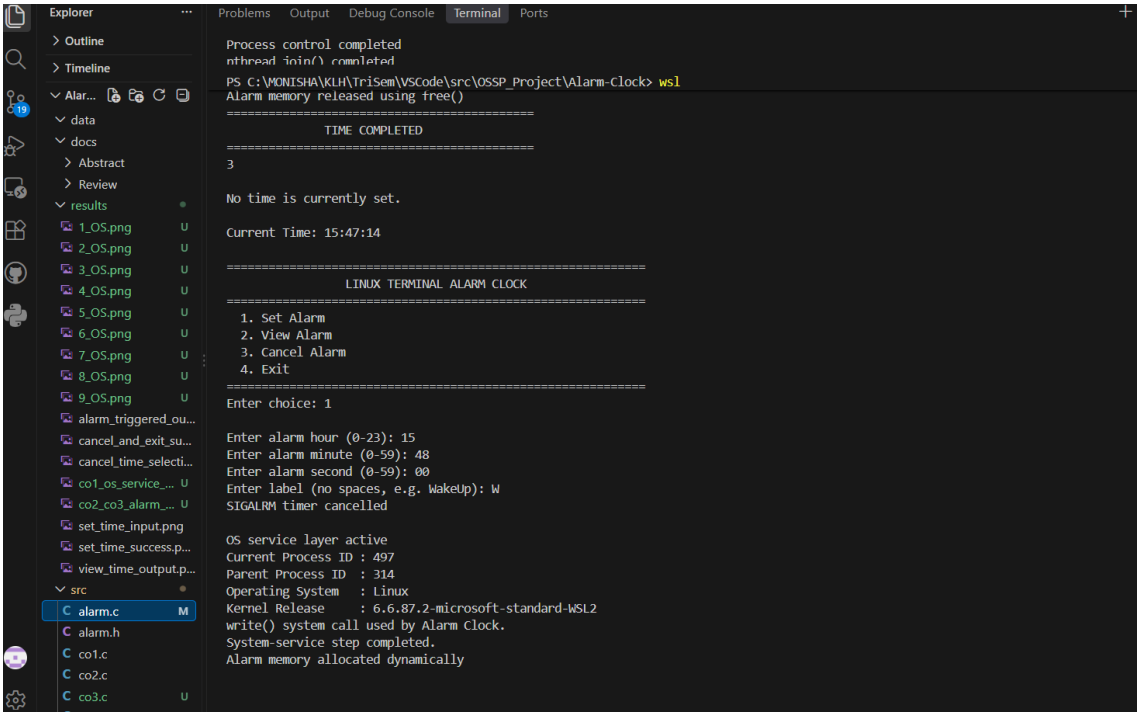


Figure 7.7: Cancel Alarm operation.

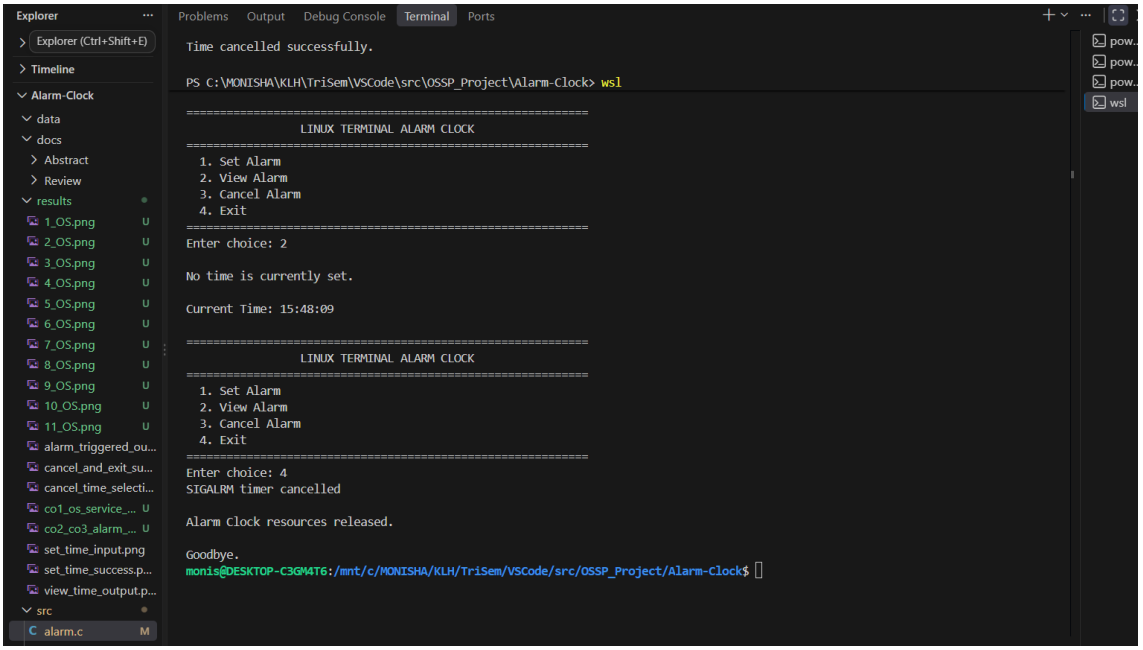


Figure 7.8: Exit operation and resource-release output.

7.5 Demonstration of OS Concepts

CO	Demonstrated Concept	Observed in Execution
CO1	OS services and system calls	PID, PPID, Linux/kernel information and write() output.
CO2	Process creation and control	Child PID, SIGTERM through kill(), and waitpid() completion.
CO3	Signals and asynchronous notification	SIGALRM timer scheduling and alarm reception.
CO4	Dynamic memory	Alarm allocation and release.
CO5	File I/O	Alarm saved, loaded or removed from data/alarms.txt.
CO6	Threads and synchronization	Monitor thread, mutex-protected state and pthread_join().

7.6 Known Implementation Limitations

The project README records two limitations that should be considered before claiming that every runtime path is fully correct. A saved alarm is loaded at startup, but its timer is not restarted automatically. The README also notes that replacing an active alarm may leave the CO6 monitor using the previous alarm state. These are documented items for future improvement.

CHAPTER - 8 CONCLUSION

8.1 Summary of the Project

The Alarm Clock project implements a menu-driven Linux terminal application in C. The application provides a simple interface through which the user can set, view, cancel, and exit an alarm.

The main purpose of the project is to demonstrate operating-system concepts through one integrated application rather than through separate programs. The project combines alarm scheduling, signal handling, process control, dynamic memory management, file storage, and thread synchronization.

The application is organized into multiple source files. main.c provides the application entry point and menu flow, while alarm.c acts as the central coordination module. The CO1–CO6 modules implement the individual systems-programming concepts.

The alarm workflow begins when the user selects **Set Alarm** and enters the required time and label. The application validates the input, calculates the required delay, and schedules the Linux alarm timer. The application then waits for the alarm event.

When the scheduled time is reached, SIGALRM is received. The alarm trigger operation coordinates the remaining processing. The project demonstrates process control through CO2, thread synchronization through CO6, file operations through CO5, and dynamic memory release through CO4.

The application therefore demonstrates how different Linux and POSIX facilities can be combined within a single C program.

The overall project flow can be summarized as:
User
↓
Alarm Clock Menu
↓
Set Alarm
↓
Validate Input
↓
Calculate Delay
↓
Schedule alarm()
↓
Wait for SIGALRM
↓
Alarm Event
↓
Process Control
↓
Thread Synchronization

↓
File Cleanup
↓
Memory Release
↓
Return to Menu / Exit
The project was developed and tested using Ubuntu through WSL, with GCC used for compilation. This provided a Linux environment in which the required signals, processes, file operations, and POSIX thread functionality could be demonstrated.

8.2 Objectives Achieved

The major objectives defined for the Alarm Clock project were achieved through the implementation and integration of the different modules.

8.2.1 Terminal-Based Alarm Interface

A terminal menu was implemented for the main user operations.
The application provides options for:
1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. Exit

This provides a simple way for the user to interact with the application without requiring a graphical interface.

8.2.2 Alarm Scheduling

The project implements alarm delay calculation and Linux SIGALRM scheduling.
The application obtains the required time information, calculates the waiting duration, and schedules the alarm using alarm().

When the timer expires, the operating system provides the SIGALRM notification that starts the alarm-trigger processing.

8.2.3 CO1–CO6 Integration

The project integrates CO1–CO6 through alarm.c.
The different components demonstrate:

- **CO1:** Operating-system services and process information.
- **CO2:** Process creation and termination.
- **CO3:** Alarm timer and signal handling.
- **CO4:** Dynamic memory management.
- **CO5:** File input/output.
- **CO6:** POSIX thread and synchronization mechanisms.

The integration of these components is one of the main outcomes of the project.

8.2.4 Process Management

The project demonstrates Linux process creation and termination using the CO2 module.

The child process is created using fork(). The parent controls the child using kill() and waits for its termination using waitpid().

This provides practical implementation of parent-child process management.

8.2.5 Dynamic Memory Management

The project implements dynamic allocation and release of the Alarm structure.
The alarm object is created when required and released when the alarm is completed, cancelled, replaced, or the application shuts down.
This demonstrates the importance of proper memory management in C.

8.2.6 File Storage

The project stores alarm information using the file:
data/alarms.txt
Linux file operations such as open(), read(), write(), close(), and unlink() are used by CO5.
This demonstrates file-based storage and cleanup within the application.

8.2.7 Thread Synchronization

CO6 implements a POSIX monitor thread. The shared alarm state is protected using a mutex, and a condition variable is used for event notification.
The monitor thread is joined after the alarm processing is completed.
This provides practical implementation of thread synchronization within the Alarm Clock application.

8.3 Results Achieved

The supplied execution screenshots demonstrate the major stages of the Alarm Clock application.

The project execution results include:

- Application startup.
- Alarm time input.
- CO1 operating-system service information.
- Alarm delay calculation and timer scheduling.
- SIGALRM event handling.
- Monitor-thread activity.
- Alarm information stored using file I/O.
- Viewing the stored alarm.
- CO2 child-process creation and termination.
- Alarm cancellation.
- Resource cleanup during exit.
-

During execution, the application displays information about the process and Linux environment.

The alarm setup stage also reports the scheduled timer and monitor-thread activity.

When the alarm time is reached, the terminal reports the SIGALRM event and performs the alarm-trigger processing.

The process-control stage demonstrates the creation and controlled termination of the alarm-related child process. The thread stage demonstrates monitor-thread completion and pthread_join().

The project also demonstrates cleanup after alarm completion. The alarm file is removed and dynamically allocated alarm memory is released.

The results therefore show the connection between the user-visible alarm operations and the underlying operating-system concepts.

The main execution stages can be summarized as:

Project Startup
↓
Set Alarm
↓
OS Service Information
↓
Timer Scheduled
↓
Monitor Thread Created
↓
Alarm Saved
↓
SIGALRM Received
↓
Alarm Triggered
↓
Child Process Control
↓
Monitor Thread Completion
↓
File Removed
↓
Memory Released
↓
Alarm Completed

The execution screenshots provide evidence for these stages and demonstrate the working behavior of the implemented modules.

8.4 Skills and Learning Outcomes

The project provided practical experience with Linux/WSL development, GCC compilation, C

modular programming, system calls, signals, process management, dynamic memory, file I/O, POSIX threads, mutexes, and condition variables.

8.4.1 Linux and WSL Development

The project provided experience in using Ubuntu through WSL as a Linux development and execution environment.
The development process included navigating to the project directory, compiling the source files, executing the generated program, and observing the Linux-specific output.

8.4.2 C Modular Programming

The project provided experience in organizing a C application into multiple source and header files.
The separation of main.c, alarm.c, and the CO modules helped demonstrate how different responsibilities can be organized within one application.

8.4.3 System Calls and APIs

The project provided practical exposure to Linux and POSIX APIs such as:
getpid()
getppid()
uname()
write()
fork()
kill()
waitpid()
alarm()
sigaction()
sigwait()
open()
read()
close()
unlink()
pthread_create()
pthread_join()
These functions demonstrate how a C application interacts with operating-system services.

8.4.4 Signals and Timers

The project provided practical understanding of the relationship between a timer and signal notification.
The use of alarm() and SIGALRM demonstrates how the operating system can notify an application when a scheduled timer expires.

8.4.5 Process Management

The CO2 implementation provided experience with creating and controlling a child process

using fork(), kill(), and waitpid().
This helped demonstrate the relationship between parent and child processes within a Linux application.

8.4.6 Memory and File Management

The project provided practical experience with dynamically allocated memory and Linux file operations.
CO4 demonstrates allocation and release of the alarm object, while CO5 demonstrates storing and removing alarm information through data/alarms.txt.

8.4.7 Thread Synchronization

The CO6 implementation provided experience with POSIX threads, mutexes, condition variables, and thread completion.
The monitor thread works with the shared alarm state and is synchronized with the alarm event before it is joined.

8.5 Overall Conclusion

The Alarm Clock project successfully combines several operating-system concepts into one menu-driven Linux terminal application.

The project demonstrates that a simple user-facing application can also provide practical implementation of systems-programming concepts. Alarm scheduling provides the central application functionality, while signals, processes, memory, files, and threads support the different stages of the alarm workflow.

The modular structure makes it possible to identify and demonstrate CO1–CO6 individually while maintaining a single integrated executable. The use of alarm.c as the central coordination layer connects the modules and allows the complete alarm lifecycle to be demonstrated.

The project also provided practical experience with Linux/WSL, GCC, C programming, system calls, signal handling, process management, dynamic memory, file I/O, and POSIX synchronization.

Based on the supplied execution results, the major functions of the application were demonstrated through startup, alarm setting, viewing, alarm triggering, process control, cancellation, and resource cleanup. The project therefore provides a practical demonstration of Linux and POSIX concepts within a single C-based Alarm Clock application.

CHAPTER - 9 REFERENCES

9.1 Books

The following books were used as background references for understanding operating-system concepts and Unix/Linux systems programming related to the Alarm Clock project.

1.Abraham Silberschatz, Peter B. Galvin and Greg Gagne, *Operating System Concepts*.

This book provides background information on operating-system concepts such as processes, threads, synchronization, memory management, and other operating-system mechanisms relevant to the project.

2.W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*.

This reference provides background for Unix/Linux programming concepts and system-level APIs. The concepts are relevant to the project's use of processes, signals, file operations, and related POSIX functionality.

These books provide the theoretical background used to understand the operating-system concepts implemented in the project.

9.2 Research Papers

The project implementation is primarily based on Linux/POSIX APIs and course-level systems-programming concepts rather than a single research paper.

The project does not depend on one specific research paper for its implementation. Instead, the development is based on the operating-system concepts covered as part of the course and the Linux/POSIX interfaces used by the application.

The main areas considered during implementation include:

- Linux process management.
- Signal handling.
- Alarm and timer functionality.
- Dynamic memory management.
- Linux file operations.
- POSIX threads.
- Mutex-based synchronization.
- Condition-variable based thread coordination.

The project combines these concepts into one terminal-based Alarm Clock application.

9.3 Linux / GNU Documentation

Linux and GNU documentation forms an important technical reference for the implementation because the project uses Linux and POSIX APIs directly.

The following documentation areas were considered:

9.3.1 GCC Documentation

GCC compiler documentation was used as a reference for compiling and linking the C source files.

The project uses compiler options such as:

-Wall
-Wextra
-pthread
-o

The compilation command used for the project is:

gcc -Wall -Wextra -pthread src/*.c -o alarm_clock

The GCC documentation provides information about compiler options, compilation, linking, and generation of executable programs.

9.3.2 Linux Process Documentation

Linux/POSIX documentation was used as a reference for process-related functions including:

- fork()
- kill()
- waitpid()

These functions are used in CO2 to demonstrate creation, control, and termination of the child process.

9.3.3 Linux Signal and Alarm Documentation

Documentation related to Linux signals and timers was used for the implementation of CO3.

The important interfaces include:

- alarm()
- sigaction()
- sigwait()
- SIGALRM

The project uses alarm() to schedule the timer and SIGALRM to provide notification when the scheduled alarm event occurs.

sigaction() and sigwait() are used as part of the signal-handling and alarm-waiting mechanism.

9.3.4 Linux File I/O Documentation

Linux file-operation documentation was used for CO5.

The project uses:

- open()
- read()
- write()
- close()
- unlink()

These operations are used to store, access, close, and remove alarm information in:
data/alarms.txt

9.3.5 POSIX Thread Documentation

POSIX thread documentation was used for CO6.

The main thread-related functionality includes:

- pthread_create()
- pthread_join()
- mutexes
- condition variables
-

The monitor thread is created using pthread_create(). A mutex is used to protect the shared alarm state, while a condition variable is used for event coordination. pthread_join() is used to wait for monitor-thread completion.

9.4 Online / Project Resources

The project also uses online and project-specific resources during development, implementation, testing, and documentation.

9.4.1 Project Source Repository

The project source repository is:

ShaikSabiyaThabassum/KLH-CSE-2026-27-16-AlarmClock

The repository contains the project source code and related project materials.
The repository was used for maintaining the project source and tracking development changes.

9.4.2 Project README and Source Files

The project README and source files provide the implementation details of the Alarm Clock application.
The main source files are:

- main.c

- alarm.c
- alarm.h
- os_project.h
- co1.c
- co2.c
- co3.c
- co4.c
- co5.c
- co6.c

These files were used as the primary implementation resources for understanding the structure and functionality of the application.

The source files demonstrate the integration of the six course-outcome components into one executable.

9.4.3 Project Execution Screenshots

The project execution screenshots stored in the results/ directory were used to document the working stages of the application.

The screenshots cover different parts of the execution, including:

- Project startup.
- Alarm input.
- CO1 operating-system services.
- CO3 alarm and signal functionality.
- CO4 dynamic memory.
- CO5 file operations.
- CO6 monitor-thread activity.
- Viewing the stored alarm.
- CO2 process control.
- Alarm cancellation.
- Resource release during exit.

These screenshots provide visual evidence of the application's execution and were used in the project documentation.

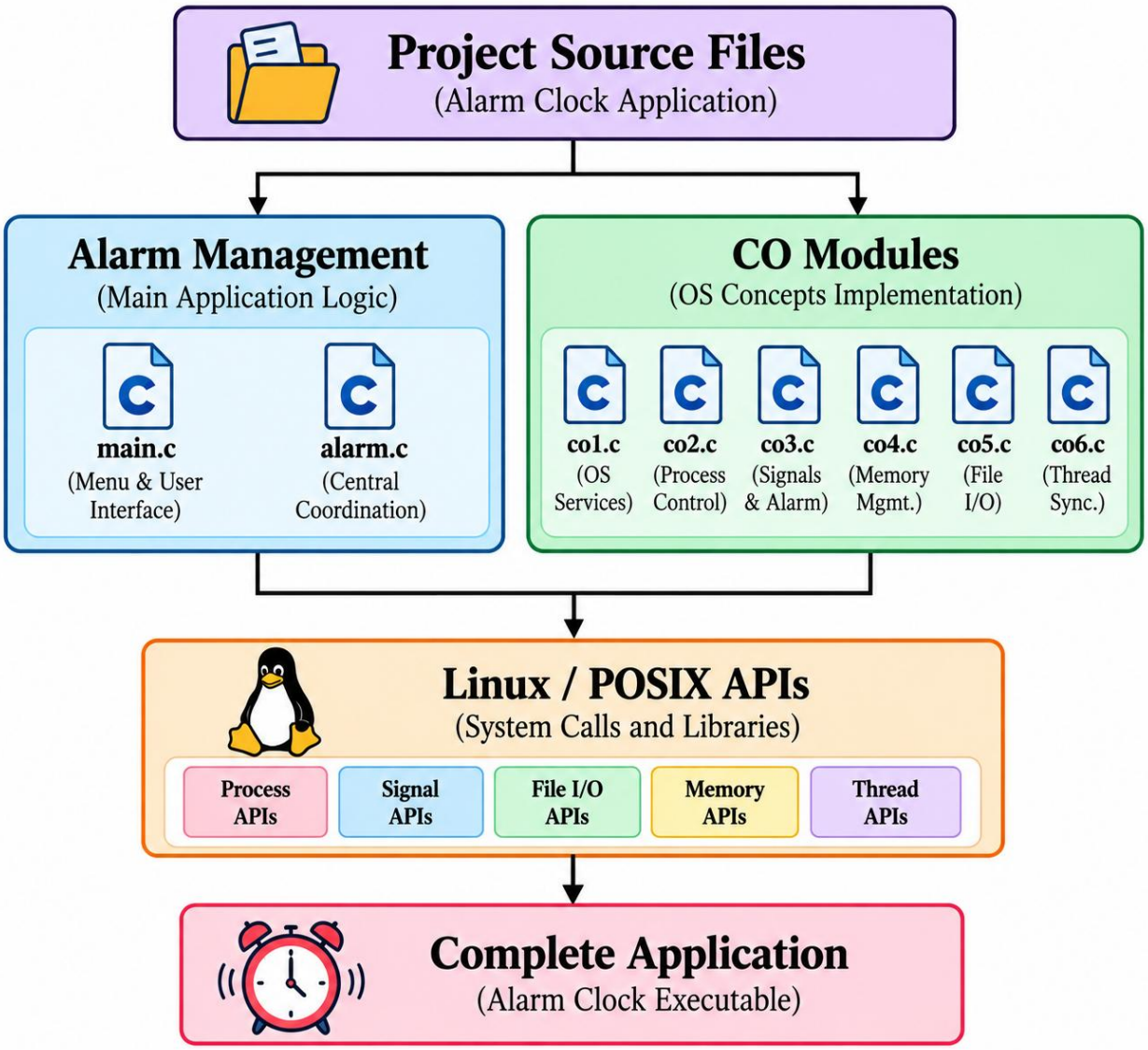
9.5 Project-Specific Reference Materials

In addition to books and Linux documentation, the project source itself serves as an important reference for understanding the implemented system.

The project header files provide the declarations required for communication between the modules.

alarm.h provides the alarm structure and public alarm-related functions. os_project.h contains declarations related to the CO1–CO6 integration.

The source files then provide the implementation of the corresponding functionality.
The relationship between the source files and the reference areas is:



This organization was useful during implementation and testing because each operating-system concept could be traced to the corresponding module.

9.6 Reference Usage in the Project

The references were used for different purposes during project development.

Reference Area	Purpose in Project
Operating System Concepts	Understanding processes, threads, memory, and synchronization
Unix/Linux Programming	Understanding system-level APIs
GCC Documentation	Compilation and linking
Signal Documentation	SIGALRM, sigaction(), and sigwait()

Reference Area	Purpose in Project
Process Documentation	fork(), kill(), and waitpid()
File I/O Documentation	open(), read(), write(), close(), and unlink()
POSIX Thread Documentation	pthread_create(), pthread_join(), mutexes and condition variables
Project Repository	Source-code management and project files
Project Screenshots	Execution evidence and documentation

These references helped connect the theoretical concepts studied in operating systems with their implementation in the Alarm Clock application.

9.7 Summary of References

The references used for the Alarm Clock project consist of operating-system textbooks, Unix/Linux programming references, Linux/GNU documentation, POSIX API documentation, and project-specific resources.

The books provide the theoretical foundation for understanding operating-system concepts. Linux and GNU documentation provides technical information about the APIs and compiler tools used during implementation.

The project repository and source files provide the actual implementation reference, while the execution screenshots provide evidence of the application's tested behavior.